



Absolute Zero

McLean, Virginia, United States

VEX Engineering Notebook

2026/2027 Override

This Page Intentionally Submitted Blank

Table of Contents

Table of Contents	3
Notebook Introduction	1
 05/04/26 Team Biography	2
 05/04/26 Team Goals	4
 05/04/26 Notebook Formatting	5
 05/04/26 Engineering Design Process	8
 05/04/26 Design Matrices & Pros/Cons Lists	9
 05/04/26 SWAT Analysis	11
 05/06/26 Code Documentation Formatting	12
 07/21/26 Time Management	13
Game Analysis	14
 08/03/26 VEX V5RC & Override Overview	15
 08/04/26 Points Scoring Breakdown	16
 08/05/26 Game Manual Analysis Introduction	17
 08/06/26 Scoring Rules Analysis	18
 08/07/26 Specific Game Rules Analysis	23
 08/08/26 Robot Skills Challenge Rules Analysis	29
 08/09/26 Other Important Rules Analysis	32
 08/09/26 Key Takeaways/Initial Strategy	33
Robot 1 "Frostbyte"	34
 08/04/26 Time Management: Robot 1	35
 08/05/26 Code Initial Setup: "12 Template"	37
 08/05/26 Identify: Drivetrain	38
 08/06/26 Research: Drivetrain Types	39
 08/07/26 Research: Wheel Sizes & RPM	42
 08/07/26 Research: Wheel Options	45
 08/08/26 Research: MoA Robots	47
 08/09/26 Research: MoA Analysis	49
 08/10/26 Final Drivetrain Options	50
 08/11/26 Evaluate & Select: Drivetrain	51
 08/12/26 Identify: Remaining Subsystems	53
 08/13/26 Final Subsystem Design Options	55
 08/14/26 Evaluate & Select: Subsystem Design	56
 08/15/26 Pre-CAD Meeting & Planning	58
 08/16/26 CAD Day 1	59
 08/17/26 CAD Day 2	60
 08/18/26 CAD Day 3	61
 08/19/26 CAD Day 4	62
 08/20/26 CAD Day 5	63
 08/21/26 Final CAD Remarks & Build Prep	64
 08/24/26 Build: Day 1	66
 08/25/26 Build: Day 2	67
 08/26/26 Test: Drivetrain Friction	68
 08/27/26 Build: Day 3	69

■	08/31/26	Build: Day 4	70
■	09/01/26	Build: Day 5	71
■	09/01/26	Test: Toggliers	72
■	09/02/26	Build: Day 6	73
■	09/03/26	Build: Day 7	74
■	09/09/26	Test: Cascade Friction	75
■	09/09/26	Build: Day 8	76
■	09/10/26	Build: Day 9	77
■	09/11/26	Build: Day 10	78
■	09/14/26	Build: Day 11	79
■	09/15/26	Build: Day 12	80
■	09/13/26	Code: Drivetrain & Intake	81
■	09/13/26	Code: Cascade	83
■	09/16/26	Build: Day 13	84
■	09/17/26	Build: Day 14	85

Appendix A: Github Repositories	86
---------------------------------	----

Robot 1	87
---------	----

Appendix B: External Sources	88
------------------------------	----

Citations	89
-----------	----

Notebook Introduction



05/04/26

Team Biography

Written By: All

Proofread By: Jasper

Proofread On: 05/05/26

Goal: Introduce the team, highlight goals and strengths, and define roles.

Jasper Majeed Hall

Age: 16**Grade:** Junior**Experience:** 3 Years

Strengths: Critical thinking, problem-solving, leadership, versatility

Primary Roles:

Designer, Coder, Builder, Notebooker

Personal Goal: Achieve the highest autonomous skills at a signature event.



Sam Kim

Age: 18**Grade:** Senior**Experience:** 5 Years

Strengths: Good at staying calm under pressure, and meeting/networking with people

Primary Roles: Driver, Builder, Scout**Personal Goal:** Drive in Dome!

Woodley Bohannon

Age: 16**Grade:** Junior**Experience:** 3 years

Strengths: Leadership, communication, innovation/creative thinking

Primary Roles: Programmer, builder, driveteam.

Personal Goal: Learn more about designing/CAD and be a top competitor at every tournament we are at.



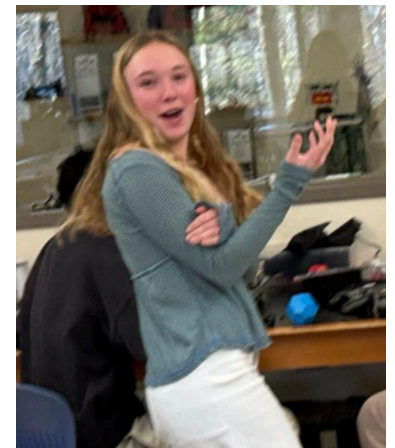
Marissa McEwen

Age: 16**Grade:** Junior**Experience:** 3 Years

Strengths: Time-management, team organization, innovative

Primary Roles: Builder, Notebooker

Personal Goal: Learn more about build/build quality and programming.



Trevor Nelson

Age: 17

Grade: Senior

Experience: 2 Years

Strengths:

Organization,
notebook,
programming

Primary Roles:

Programmer,
Notebooker, Builder

Personal Goal: Win a judged award or Skills
Champion at a signature event or states.



Helena Backus

Age: 16

Grade: Junior

Experience: 3 Years

Strengths:

Organization,
leadership,
communication

Primary Roles: Scout,

Notebooker, Builder

Personal Goal: Learn
more about programming.





05/04/26

Team Goals

Written By: Trevor

Proofread By: Jasper

Proofread On: 05/05/26

Goal: Establish long-term goals and milestones to help guide our season.

We want to set specific goals and milestones, so we can keep track of our progress throughout the season, and make sure we're meeting our goals. Due to our large number of goals this season, we've split them up into the 6 main categories seen below.

Team Dynamic/Structure

- Foster a **close relationship** between all team members (particularly since we've added 2 new members this year, Sam and Trevor).
- Ensure that all team members have input on decisions, and **feel valued**.
- Play to the **strengths of all members**, and ensure they're in the roles they fit best.

Programming

- Keep our codebase simple and easy to navigate and understand.
- Implement the use of **sensors** to improve consistency.
- Achieve **>90% consistency** for autonomous routines and programming skills.
- Ensure that we have autonomous routines that **complete the AWP criteria** for both local and Signature Events.
- Develop "12 Temp" as an easy to navigate template for new teams in our organization.

Notebooking

- Create entries that contain **detailed documentation** of our design process.
- Maintain **consistency** with our entries.
- Document all important details in a timely manner.
- Proofread all entries within 1-2 days.

Match Play

- Consistently **practice driving**.
- Ensure that our **drive team members are all cohesive** so that good calls can be made during matches.
- Ensure that our **strategy adapts** to how match play evolves throughout the season.

Design Process

- Design a robot that fits the preferences and driving style of our driver.
- Thoroughly **design our robots in CAD** before building.
- Employ **effective time management** throughout all parts of the design process, as well as programming.

Tournament Performance

- Win at least **75%** of our matches.
- Win a banner at a Signature Event or States (Excellence, Tournament Champions, Design).
- Win a **judged award** at a sig/States and Worlds.
- Make it to our Division Finals at Worlds.



05/04/26

Notebook Formatting

Written By: Trevor

Proofread By: Jasper

Proofread On: 05/05/26

Goal: Create a reference for our notebook formatting for team use throughout the season and to help non-team members, such as judges, understand how to read our entries.

General Page Setup

This notebook format is inspired by [2654E Echo's notebook formatting tutorial](#) about how to properly organize a notebook. While we have adapted elements of their page design, our approach to documenting the design process is developed collaboratively by our team and reflects a blend of our own experiences.

- **Entry Title:** The title of every entry will contain the date the entry was written on and a very brief description of what it is about.
 - Titles for design process entries will be under the format (Design Process Stage: Entry Title).
 - Entry titles will always appear at the top of a page. New entries will always begin on the next blank page following the end of the previous entry.
- **Entry Bar:** This bar always appears below the entry title and includes the entry's contributors and the date the entry is finalized.
 - "Written By" means the person or persons who wrote the entry. The writer or writers of the entry do not have to be those who worked on the content that is being documented.
 - "Proofread By" means the person who proofread the entry and ensured the documentation is accurate. Multiple team members can do this.
 - "Proofread On" means the date the entry was proofread and finalized. This date should commonly be the same date the entry was written, but at the latest, the following day.
- **Goal:** A short statement about goals for the respective entry.
- **Footer:** Our footer contains "12A Absolute Zero", which online readers can click to instantly go back to the table of contents.

Documentation Guidelines

Will be Documented	Will <u>Not</u> be Documented
● Any significant changes in the design or programming of our robots ● Game analysis and strategic discussion throughout the season, usually after tournaments and game manual updates ● Testing and reflections about robot subsystems ● Time management, with the use of calendars and Gantt charts	● Very minor changes in build, code, etc. ● Cosmetic robot decorations or other elements that don't relate to the functionality of our robots ● The bulk of our programming, as we anticipate several thousand lines of code ○ However, any significant pieces of code (macros, controller layout, etc.) WILL be documented

Section Color Key

This year we will implement a key, with each color representing a different type of entry (found below). This color will be found for every entry in the table of contents, and in the title. These will be helpful for quickly and easily identifying the topic about which we are writing. Additionally, while viewing online, **if you click on the colored square it will take you to this page** so you can view the key easier.

Color:	Topic:
	Red: Game Analysis & Game Strategies
	Orange: Evaluate & Select
	Yellow: Programming
	Green: Weekly Updates
	Blue: Identify & Brainstorming
	Purple: Notebook Setup, Organization & Timelines/Gantt Charts
	Brown: Testing
	Black: Research & Tournament Analysis

Note: All citations for any sources (images, videos, etc.) that we have used can be found in [Appendix B](#).

Example Page

On the next page we will have an example outlining how a typical page will look. Please note this is not an actual page that will be found in our table of contents, but simply a representation of our formatting.

Topic Color
(Click to view key)



Date
05/04/26



Section Title
Notebook Example



Title



Written By: [Writer]	Proofread By: [Proofreader]	Proofread On: [Proofread Date]
----------------------	-----------------------------	--------------------------------

Goal: [Statement here]

Subheading
Text

Section Color



Example Chart	
Pros	- Major Pro 1 - Major Pro 2 - Major Pro 3 - Pro 1 - Pro 2 - Pro 3
Cons	- Major Con 1 - Major Con 2 - Major Con 3 - Con 1 - Con 2 - Con 3

Footer to return to
Table of Contents



Page Number





05/04/26

Engineering Design Process

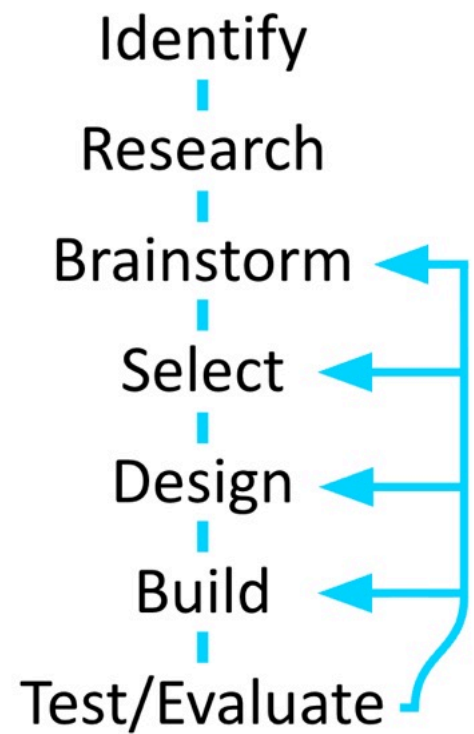
Written By: Trevor

Proofread By: Jasper

Proofread On: 05/05/26

Goal: Describe the design process and explain how we intend to use it.

- **Identify:** Clearly and accurately outline the nature of the challenge or problem that must be solved; we cannot fix a problem unless we know what the challenge is.
- **Research:** Gather relevant data, information, and ideas from various sources, such as other teams, robots designed for previous games, and live streams of competitions. This inspiration will ensure that we remain competitive against the highest-level teams.
- **Brainstorm:** Generate at least three concepts or ideas that fulfill our criteria and analyze their competence in all areas of our criteria.
- **Select:** Compare our brainstormed solutions within a table using well-outlined design matrices to select which will be implemented on our robot. Different criteria will be weighted more than others.
- **Design:** Utilize CAD to create the aforementioned solution. This planning prior to building helps to mitigate any unforeseen issues or complications that may arise throughout the building process.
- **Build:** Construct the robot or subsystem as it was outlined in the CAD.
- **Test/Evaluate:** Rigorous assessment and methodical examination of the robot or subsystem through various tests and trials to identify any unforeseen problems or oversights. We must identify any issues we have in order to ensure our robot is as effective as possible.



On most occasions, the design process resembles a non-linear path, but a complex web of these steps. When an issue is found during testing, it means that we need to **go back to a previous step** and reevaluate what we are doing. Sometimes, if there is a small issue, we may only need to tweak the design or build portion slightly, but with larger issues, it may be necessary to either **select an alternative solution** or brainstorm altogether new solutions. This **idea of a web** rather than a linear path cements within us the idea that we must be **open to failing and reworking our ideas** in order to not only build a better robot, but also **learn what it means to fail** and know how to react when that happens.



05/04/26

Design Matrices & Pros/Cons Lists

Written By: Trevor

Proofread By: Jasper

Proofread On: 05/05/26

Goal: Explain the importance of effectively comparing design options using Design Matrices and Pros/Cons lists.

Understanding the advantages and disadvantages of a concept is incredibly important when weighing different ideas. One way to do this is a simple pros and cons list, and the other is a more advanced design matrix that weights criteria to help make decisions. Below we will illustrate our approaches for them this season.

Design Matrices

Definition:

Design Matrices are tools that allow for the comparison of different design options, based on a number of weighted criteria, which we will determine based on the subsystem in question. Ours throughout the season will use a varying number of criteria, based on the complexity of the concept we are evaluating.

Example Design Matrix:

Criteria:	Weight	Choice 1		Choice 2		Choice 3	
		Score	Weighted Score	Score	Weighted Score	Score	Weighted Score
Criteria 1			0		0		0
Criteria 2			0		0		0
Criteria 3			0		0		0
Criteria 4			0		0		0
Total Score:			0		0		0

Uses:

We will typically use design matrices for advanced or complex parts of our robot (drivetrain, intake, etc), or when there are 3 or more designs to evaluate. We'll use different criteria for each, depending on what we wish to focus on, but the structure and use case will be similar. Each score and weight are an integer 1-10, and the total score is a sum of the weighted scores. The weighted scores are the product of the score and its categorical weight. The highest total score is the chosen design we will proceed with.

Benefits:

- Comprehensive Comparison
 - Allows for comparison of **numerous concepts/designs**
 - Weights criteria to ensure effective scoring and evaluations
- Identifies Key Trade-offs
 - Helps to identify trade-offs, by using **multiple weighted criteria**. For example, it can compare speed, weight, and durability all at the same time, and make decisions easier and clearer.

Pros/Cons Lists

Definition:

Pros/Cons Lists are simple, yet useful evaluation methods that compare the advantages and disadvantages of a concept. These lists typically evaluate a single design, but can also be used in tandem with other pros/cons lists to evaluate the overall merit of the concepts.

Uses:

We'll typically use pros/cons lists if there are only **1-2 design options** we're considering, or if the design is relatively simple. We may also use these in tandem with design matrices, in order to determine our overall choice, and the individual merits of a design.

Importance:

- Simple Decisions
 - Given their previously discussed simplicity, it can make it a lot easier to evaluate decisions, especially for smaller subsystems.
 - The simplicity also makes it a lot more intuitive, and can allow readers to better understand our reasoning behind our choices.
- Flexibility
 - Pros/Cons lists are easily adaptable to different situations, and rather than relying on specific criteria we select, they can simply list the objective advantages and disadvantages.

Example Pros/Cons List:

<u>Example Design Option</u>	
Pros	● Major Pro 1 ● Major Pro 2 ● Major Pro 3 ● Pro 1 ● Pro 2 ● Pro 3
Cons	● Major Con 1 ● Major Con 2 ● Major Con 3 ● Con 1 ● Con 2 ● Con 3

Conclusion

Both Design Matrices and Pros/Cons lists are vital tools when it comes to deciding an approach or design to take. Design matrices offer comprehensive, in-depth analysis, making them ideal for bigger decisions that have many design options and criteria. Pros/Cons lists are simple and straightforward, making them ideal for smaller scale decisions with less design options and criteria. Using these methods, we can make **informed decisions that balance various factors** and lead to choosing the most suitable designs for our team.



05/04/26

SWAT Analysis

Written By: Trevor

Proofread By: Sam

Proofread On: 09/20/26

Goal: Explain what “SWAT” Analysis is, along with the benefits and use cases this season.

What Is SWAT Analysis?

SWAT (Strengths, Weaknesses, Analysis, and Timeline) is a method used in numerous industries to evaluate projects, and it has been adapted slightly (Analysis and Timeline differ from the original) to better fit our use case in Vex robotics. This method was introduced to us by Tony Diller, who was an incredible notebooker on our sister team 12Z last year. The key to a SWAT analysis is to **look objectively at performance** and see what needs to change and what needs to stay the same.

The order in which we conduct this process is important. We start with “successes” as it allows us to have a positive outlook on the rest of our analysis. We then go to “weaknesses”, which allow us to critically and specifically identify mistakes that contributed to our performance. We then use “analysis” to brainstorm what we can implement or fix in order to not make those same mistakes. The “timeline” is used to plan if we can implement our analysis within the time frame of the next tournament.

Uses/Benefits

SWAT will allow us to effectively evaluate our performance after tournaments, and see how we can improve. The four aspects of the strategy are important, because it allows us to **comprehensively evaluate our decisions and performance**. SWATs will be written by all team members so we can make sure everyone’s issues can be addressed and strengths identified.

Example SWAT Analysis

Successes:

- **Success #1**
 - Evaluations
- **Success #2**
 - Evaluations

Weaknesses:

- **Weakness #1**
 - Evaluations
- **Weakness #2**
 - Evaluations

Analysis/Proposed Solutions:

- **Analysis/Solution #1**
 - Evaluations
- **Analysis/Solution #2**
 - Evaluations

Timeline:

- **Upcoming Events/Schedule #1**
 - Evaluations
- **Upcoming Events/Schedule #2**
 - Evaluations



05/06/26

Code Documentation Formatting

Written By: Trevor

Proofread By: Helena

Proofread On: 5/07/26

Goal: Outline our mentality and structure for documenting our programming and code base.

What We'll Document

When it comes to documenting code, there are many approaches that can be taken. During my (Trevor) first year, my team (12C) had a separate notebook dedicated entirely to programming and included every line of code we had. However, we quickly realized that wasn't optimal because it required way more time and it made it very hard for judges to understand. So, based on that (and the fact that the whole team has experience with this method) we've decided to **demonstrate key code elements and summaries** while **omitting unnecessary content**. What constitutes key elements vs unnecessary content, you may ask? Well, please refer to the chart on the right to see our beliefs. Though please note this is simply a general guideline and some exceptions may occur as we see fit.

<u>Included vs Omitted Content</u>	
Include	• Version control information • Key subsystem control methods • Auto paths • Controller button programming
Omit	• Complete code for autos • Cosmetic programming (auto selector, etc.)

Documentation Structure

Our structure for documentation will typically include the relevant portion of code, and then a description of what it does. We will include pros and cons associated with the code, and any next steps that need to be taken. Additionally, we will document all autonomous routes. To do so, we will draw out the path on a photo of the field, list the benefits of the path, discuss any errors, and plan next steps. When programming an autonomous, we will **document every run from the very start**. This will allow us to note how long it typically takes to write an auto, see where inconsistencies may lie, and later use that data to influence future autonomous routes.



07/21/26

Time Management

Written By: Trevor

Proofread By: Helena

Proofread On: 7/22/26

Goal: Explain the importance of effective time management throughout the season and the role of Gantt Charts.

Importance of Time Management

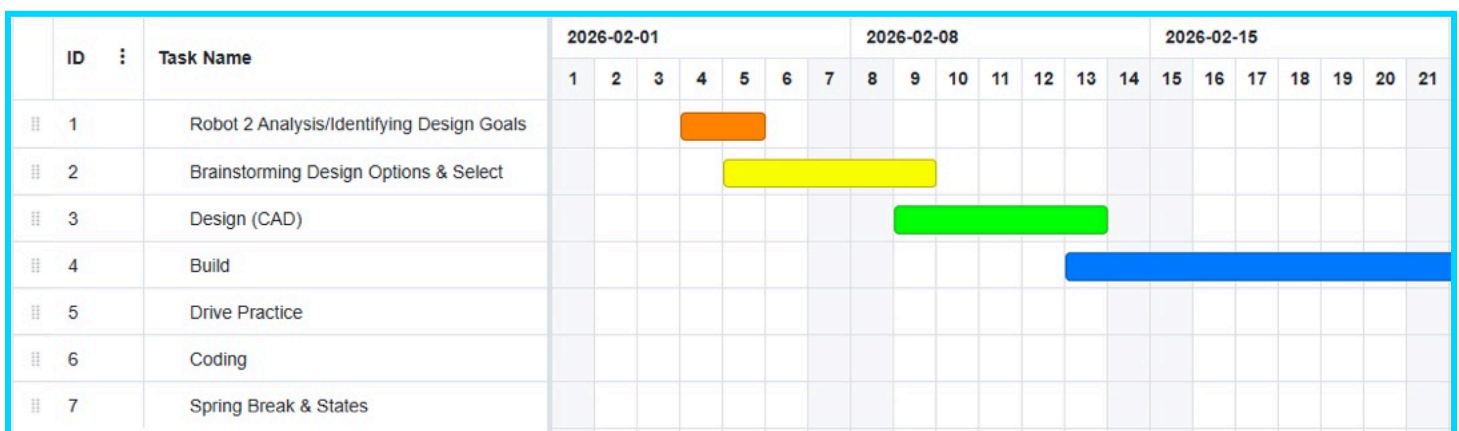
Something we hope to do this season is really be on top of our time management. In early team meetings, we realized that on our previous teams, time management wasn't always a strong suit, with many missed deadlines and poor planning. So, this year **we all wish to collectively grow in that regard**. The main way we will do this is by implementing Gantt Charts and ensuring accountability within the team.

This summer I (Trevor) had the opportunity to do a summer internship at Clark Construction Group, and while there I learned a lot about time and personnel management while at a job site. While there, I learned a lot about scheduling including how to use Gantt Charts and how to find critical paths that allow for float. Now, I wish to **use what I learned there and apply it to our team this year**.

Gantt Charts

A Gantt Chart is a visual time management tool that allows events to be overlapped while easily viewing progress. This tool is extremely useful in competitive robotics, as numerous things can be happening at once. When programming, building, notebooking, and more all need to happen in the same day, Gantt Charts make it simple to view and understand what needs to be done. Below is a portion of an example Gantt Chart from last year's team 12C.

Example Gantt Chart:



Game Analysis



08/03/26

VEX V5RC & Override Overview

Written By: Trevor

Proofread By: Helena

Proofread On: 08/04/26

Goal: Explain the most crucial aspects of the VEX V5RC competition and the 2026/2027 game, Override.

VEX V5RC Competition Overview

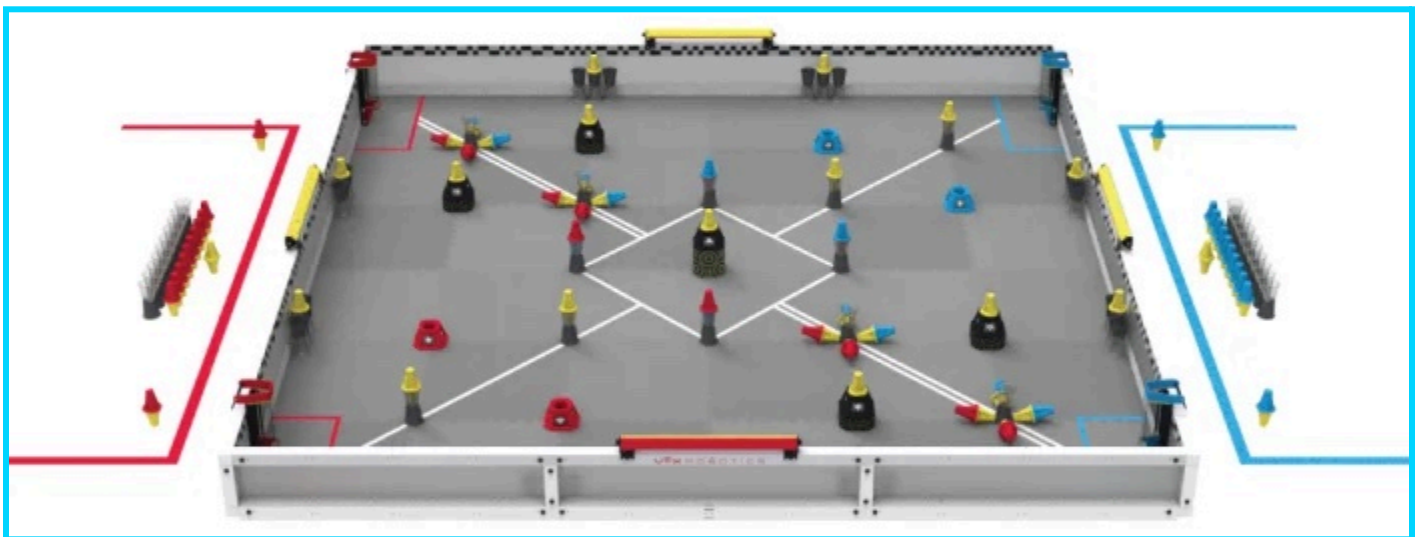
Each year, the VEX V5RC robotics competition releases a new game. These games have two main components: the skills challenge and the regular tournament.

The tournament portion of the competition is made up of matches. In these matches, teams try to maximize the number of win points they can get to increase their ranking. Win points (WPs) can be obtained in two main ways: meeting the criteria for the autonomous win point during the 15-second autonomous period (1 WP), winning (2 WPs), or tying a match (1 WP). A team wins a match if they and their alliance partner score more points than the opposing alliance within the allotted time.

In skills, robots compete independently to maximize their points. At each event, teams are ranked based on their combined highest programming and driver skills scores in one-minute robot skills matches.

Override Overview

This year's V5RC game, Override, is played on a 12' x 12' square field with 2 red goals, 2 blue goals, 5 neutral goals, 4 match loaders, and 4 toggles. The game has 56 cups, each with one clear and one gray side. It also has 4 red/blue pins, 20 red/yellow pins, 20 blue/yellow pins, and 19 yellow/yellow pins, 63 total. Cups and pins can be found positioned around the field or as match loads for the alliances. At the end of the game, a "king of the hill" style challenge occurs in the four center tiles.





08/04/26

Points Scoring Breakdown

Written By: Trevor	Proofread By: Jasper	Proofread On: 08/04/26
--------------------	----------------------	------------------------

Goal: Identify how points can be scored during a match; discuss individual point values, and the points that can be achieved by scoring in that specific manner, the maximum number of times.

Name	Individual Point Value	Maximum Possible Points
Autonomous Bonus	12	12
Alliance Colored Pin Scored	5	100
Owned Yellow-Yellow Pin	10	190
Owned Yellow-Red Pin	10	200
Owned Yellow-Blue Pin	10	200
Robot in the Midfield	8	16

*Note there are only 56 cups, so the maximum total points from purely pins would be 560 points.

To create this table, individual point values were sourced directly from the [Override Game Manual](#). To find the maximum points possible, we multiplied the individual point values by the number of opportunities available to score each one, which we found in the game manual as well. Our analysis leads us to believe that one of our main focuses should be **programming reliable autons** so that we may receive the 12-point bonus, the AWP (autonomous win point), and the strategic advantages of having an auto that leaves you in a beneficial position over opposing alliances.

In addition to autonomous routines, we feel it will also be important to be able to quickly and easily rotate the toggles, given that the yellow pins (worth 10 points each) are only scored for our alliance if the toggle matches our color. Hence, we'll need to be able to **easily gain control over the toggles**.

Finally, we'll also want to have a bot that can easily **withstand pushing** from any direction, so we can remain in the midfield and win the king of the hill type endgame. It's not mentioned here since it doesn't directly score any points (we'll go over it later in our game manual analysis though), but the alliance with the most robots in the midfield gains control over all yellow pins at the end of the game. Given this, the alliance with the most bots can gain 8 or 16 points (depending on the number of bots in the midfield), and 10 points per yellow pin. So, we think it will be **very important to be able to stay in control of the midfield**.



08/05/26

Game Manual Analysis Introduction

Written By: Trevor

Proofread By: Jasper

Proofread On: 08/05/26

Goal: Create a template and ideology for how we will analyze key rules from the [Override Game Manual](#).

Rule Analysis Template

Rule:

We'll place the unedited rule from the game manual here. On occasion we may omit red boxes or violation notes if they're deemed unnecessary.

Interpretation:

Here we'll write out how we interpret the rule. These interpretations will be based on the **specific language of the manual and any knowledge about the rule** that we may have from previous years. If a rule is extremely self-explanatory we may not add an interpretation, but in general we will write one.

Impact:

In this section we'll write about how it may impact either game strategy, build decisions, tournament performance, or anything else in the engineering design process. As with interpretations, these impact assessments will be based on our own knowledge, along with outside resources such as game manual breakdown videos, and knowledge of previous games. We will attempt to be as comprehensive as possible in our impact evaluations, so when starting to build our first robot and go to tournaments, we can understand the game as much as possible.

Rules We'll Analyze

Last year we analyzed every single rule in the game manual, but we found that to be extremely time consuming and rather unnecessary. A majority of the rules (especially safety rules, inspection rules, tournament rules, etc.) stay the same, so **it's unnecessary to reevaluate most rules** at the beginning of the season. That being said, the entire team will read through the manual, and if there are any changes, we'll evaluate those. We'll also evaluate any that seem especially pertinent to this year's game, but we expect to **focus primarily on game-specific rules** (such as scoring rules, skills rules, and specific game rules).



08/06/26

Scoring Rules Analysis

Written By: Trevor

Proofread By: Helena

Proofread On: 08/06/26

Goal: Analyze all Scoring Rules in the [Override Game Manual](#) to cement our understanding of the rules and identify any key impacts the rules may have.

<SC1> All scoring statuses are evaluated after the Match ends. Scores are calculated five seconds after the *Match* ends, or once all *Scoring Objects*, *Field Elements*, and *Robots* on the *Field* come to rest, whichever comes first.

- a. This 5-second delay is intended to be the only permitted “benefit of the doubt” for last-second scoring actions. If an object or *Robot* is still in motion and “too close to call” between two states at the 5-second mark, then the less advantageous of the two states should be awarded to the *Robot(s)* in question. A *Robot* which is breaking the plane of the *Midfield* but slowly droops down and away from the *Midfield* at five (5) seconds would not be considered in the *Midfield*.
- b. At the end of the *Match*, the on-screen timer displayed by Tournament Manager will hold the current *Match* information and “0:00” for five (5) seconds before moving to queue the next *Match*. This should be the primary 5-second visual cue used by *Teams* and *Head Referees*.
- c. This 5-second delay is only intended to be a “benefit of the doubt” grace period, not an extra five seconds of *Match* time. *Robots* which are designed to strategically exploit this grace period will receive a *Minor Violation*, and any post-*Match* movement will not be included in score calculation (i.e., the *Match* will be scored as it was at 0:00).
- d. Referees should avoid contacting or moving *Robots* and/or *Scoring Objects* as much as possible while evaluating scoring statuses. If an object must be moved to evaluate the status of another object, its status must be agreed upon by all *Teams* and the *Head Referee*, and noted or recorded, before it is moved.
- e. Referees must record counts based on verified scoring statuses evaluated after the *Match*, using final positions of *Scoring Objects*, *Field Elements*, and *Robots*. Point considerations used to determine whether a *Violation* is *Match Affecting* (e.g., specified in *Violation Notes*) should NOT be added to or deducted from the actual score, and points scored during a *Violation* should not be deducted from a score.

Interpretation: This rule primarily ensures that all cones and cups are securely placed, and no towers will fall. Additionally, this means a robot in the midfield has to remain in the center for the 5 seconds after too (though unlikely, we suppose a fast moving robot could drift out when stopped at the end of the match).

Impact: Last year this rule was very important given the high-speed pace of the game, and the tendency for blocks to roll. This year we expect the game to be a lot slower pace, and more strategy and precision based. Hence, we don’t think this rule will have a super high impact, but it is important to ensure our bot is stationary at the end of a match if we’re trying to win the midfield, so as not to lose the bonus and lose control of the yellow cones.

<SC2> A Pin is considered Placed if it meets all of the following criteria at the end of the *Match*:

- The *Pin* is partially or entirely nested with a *Goal*, or with a *Cup* that is partially or entirely nested to another *Placed Pin*.
- That half of the *Cup* only contains one *Pin*.

In the context of <SC2>, nested means that one half of a *Pin* is partially or completely contained within the inner volume of the *Cup*. In other words, the *Pin* is breaking an imaginary plane at the opening of a *Cup*, in any way. *Robot* contact does not matter for the purposes of <SC2>, and a *Robot* that is contacting or possessing a *Pin* that still meets the criteria presented in <SC2> does not negate the *Placed* status, provided no other rules are broken (particularly <SG6>).

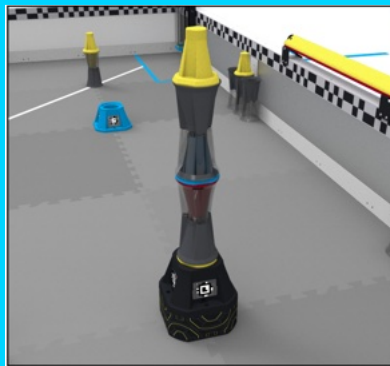


Figure SC2-1 These Pins all count as Placed, as they are all at least partially nested within each other, and are not being contacted by any Robots.



Figure SC2-1 This Pin is not Placed, as it is not nested within a Cup or Goal.

Interpretation: Cups may only contain **one pin**, and that pin must be nested (breaks the plane of the cup or goal in which it is placed) in order to be considered placed. Additionally, pins that a robot is touching still count as placed.

Impact: Last year during Push Back, blocks we were touching at the end of a match did not count, but this year our robot can touch them. This is very important, because it means at the end of the autonomous period and the match, we won't have to worry about pulling away from a block to not touch it. Additionally, the **pin only needs to break the plane of the cup/goal** to be scored, so we can simply hold a pin in a cup/goal, and it will still count.

<SC3> Each *Pin* consists of two halves. **Each Placed Pin can have one or two Scored halves.** A half counts as *Placed* for the corresponding *Alliance* if it is either nested inside the transparent half of a *Cup* or is not covered by a *Cup*.

- a. A *Placed Pin* displaying the color red is scored for the red *Alliance*.
- b. A *Placed Pin* displaying the color blue is scored for the blue *Alliance*.
- c. A *Placed Pin* displaying the color yellow is scored for the *Alliance* that *Owns* the *Pin* (see <SC5>).

Interpretation: Alliance colored pins are scored for that alliance, and yellow pins are scored for whoever owns them. Either or both halves of a pin can be considered scored if not covered by a cup, or covered by the transparent half of a cup.

Impact: Pins covered by the opaque side of a cup will not be counted, so we can place the opposite alliance's color into those halves, and therefore reduce the number of points they can score. Additionally, a pin does not need a cup on top to be scored, so we can place just pins if there's an opening, and those points will still count.

<SC4> A Toggle is considered set to a color when it meets all of the following criteria:

- a. The *Toggle* must be fully seated, such that there is a face of the *Toggle* in contact and parallel with its mounts on the *Field Perimeter* at rest. (see Figure SC5-1)
- b. The *Toggle* is not in contact with a *Robot* from either *Alliance*.

If a *Toggle* is not considered set to a color, it is considered a neutral (yellow) *Toggle* by default, and neither *Alliance* receives *Ownership* of the yellow *Pins Placed* in that *Quadrant*. While the *Toggle* has infinite potential orientations, only three discrete orientations are considered "set" states.

Interpretation: The toggle must be fully seated and not contacting a robot.

Impact: The most important part seems to be part b, as it means we have to ensure that we're not contacting the toggle at the end of the match. Although, we may be able to use this to our advantage. If the toggle is set to the opposing alliance and we don't have time to flip it to our color, we can simply contact the toggle to negate their points.

<SC5> Yellow Pin Ownership. Each *Placed Pin* with one or more yellow halves can be *Owned* by an *Alliance*.

- a. A yellow *Pin Placed* in a *Quadrant* is *Owned* by an *Alliance* if the *Toggle* in that *Quadrant* is set to the *Alliance's* color. If the *Toggle* is set to yellow, *Placed yellow Pins* in that *Quadrant* are not *Owned* and do not score points. (See <SC3>.)
- b. A yellow *Pin Placed* in the *Midfield* is *Owned* by the *Alliance* that ends the *Match* with a greater number of *Robots* in the *Midfield*. (See <SC6>.)
 - i. If both *Alliances* end the *Match* with an equal number of *Robots* in the *Midfield*, yellow *Pins Placed* in the *Midfield* are not *Owned* and do not score points.

Interpretation: Yellow pins are only owned if the alliance owns that quadrant or has a majority number of robots in the midfield. If a toggle is yellow or the number of robots are tied, yellow pins count for nothing.

Impact: We'll need to ensure that toggles are set to our alliance color if we wish to obtain the points from yellow pins. At the end of the match we'll also have to focus on placing our robot in the midfield to gain control of the central yellow pins.

<SC6> A Robot counts as ending in the Midfield if the *Robot* is at least partially within the *Midfield* at the end of the *Match*.

Interpretation: If any part of the robot crosses the midfield line, it counts as ending in the midfield.

Impact: Our robot doesn't have to end entirely inside of the midfield to qualify for control. So, we can simply place a corner of the robot into the midfield in order to gain control of the yellow pins.

<SC7> Scoring of the Autonomous Bonus is evaluated immediately after the *Autonomous Period* ends.

- a. Scoring methods that rely on *Robots' end-of-Match* positioning in the *Midfield* (e.g., *Robots* ending in the *Midfield* and *Ownership* of yellow *Pins* in the *Midfield*) are not included in *Autonomous Period* scoring calculations.
- b. If the *Autonomous Period* ends in a tie, including a zero-to-zero tie, each *Alliance* will receive an *Autonomous Bonus* of six (6) points.
- c. Any *Violations*, Major or Minor, committed during the *Autonomous Period* will result in the *Autonomous Bonus* being automatically awarded to the opposing *Alliance*. See <GG13>.
- d. Per rule <GG13>, if both *Alliances* commit *Violations* during the *Autonomous Period*, then no *Autonomous Bonus* will be awarded.

Interpretation: The 12 point Autonomous Bonus can only be won by gaining more points than the opposing alliance and committing no violations.

Impact: We must ensure our autos don't break any rules and achieve the maximum points to win the bonus.

<SC8> An **Autonomous Win Point** is awarded to any *Alliance* that ends the *Autonomous Period* with all of the following tasks completed, and that has committed no *Violations* during the *Autonomous Period*:

1. At least seven (7) *Pins Scored* for your *Alliance*. (Does not include *Pins Scored* in *Quadrants* on the opposing side of the *Autonomous Line*.)
2. At least three (3) *Goals* each contain at least two (2) *Pins Scored* for your *Alliance*. (Does not include *Goals* in *Quadrants* on the opposing side of the *Autonomous Line*.)
3. Neither *Robot* is contacting the *Field Perimeter*

Interpretation: Pins scored by the opposing alliance do not count towards our AWP. 3 goals must contain 2 or more pins, adding up to 7 in total to win the AWP. Our robot cannot end the auto touching the field wall.

Impact: We'll need to ensure we meet all of these criteria in order to secure the AWP. If we create an autonomous route that secures the AWP with only our robot, we'll need to ensure our alliance at least moves off of the wall.



08/07/26

Specific Game Rules Analysis

Written By: Helena

Proofread By: Trevor

Proofread On: 08/08/26

Goal: Analyze all Specific Game Rules in the [Override Game Manual](#) to cement our understanding of the rules and identify any key impacts the rules may have.

<SG1> Starting a Match. Prior to the start of each *Match*, each *Robot* must be placed such that it meets all of the following criteria:

- a. No larger than 18" (457.2 mm) long by 18" (457.2 mm) wide by 18" (457.2 mm) tall.
- b. Not contacting any *Scoring Objects* other than a maximum of one (1) *Preload*. See rule <SG5>.
- c. Not contacting *Goals*, *Loaders*, *Load Zones*, or *Toggles*.
- d. Not contacting any other *Robots*, and not sharing a *Quadrant* with another *Robot*.
- e. Completely stationary (i.e., no motors or other mechanisms in motion).
- f. Contacting the *Field* tiles and *Field Perimeter* on their *Alliance's* side of the *Autonomous Line*.

Interpretation: Our robot must meet specific criteria to be considered legal at the start of a match. It must fit within the 18" × 18" × 18" starting size limit, remain entirely on our side of the field, and cannot contact any field elements or game objects other than one preload.

Impact: We will design our robot so that it remains within the 18" size limit at the start of each match while still allowing us to use a preload effectively. Before every match, we will also verify that the robot is fully on our side of the field and is not contacting any prohibited field elements or game objects.

<SG2> Horizontal expansion is limited. Once the *Match* begins, *Robots* may expand horizontally beyond the starting size limit within the following criteria:

- a. The *Robot* can never be larger than 24" wide or 24" long at any point during the *Match* (must fit within a 24" x 24" square horizontal footprint).

Interpretation: Our robot may expand beyond its 18" starting size horizontally during a match, but it cannot exceed 24" in either width or length.

Impact: When designing and CADing our robot, we will carefully check its dimensions to ensure that it does not exceed the 24" expansion limit. Additionally, since the robot can extend so high vertically, we will need to ensure it does not sway/tip at the top and accidentally put us over the limit.

<SG3> Vertical expansion is limited. Once the *Match* begins, *Robots* may expand vertically beyond the starting size limit, but no part of the *Robot* may exceed an overall height of 50" at any point during the *Match* (must always be able to fit within a hypothetical 50" vertical sizing box). For exceptions to this rule involving *Robots* in the *Midfield* during the *Endgame* period, see <SG12>.

Interpretation: Our robot may expand beyond its 18" starting height during a match, but it cannot exceed 50" vertically at any point.

Impact: When designing and CADing our robot, we will carefully check its dimensions to ensure that it does not exceed the 50" vertical expansion limit.

<SG4> Keep Scoring Objects in the Field. *Teams* may not intentionally remove *Scoring Objects* from the *Field*. A *Scoring Object* that leaves the *Field* during *Match* play will be returned to the *Field* in a location near where it left, in contact with the *Field* tiles and the *Field Perimeter* but no other *Field Elements* or *Scoring Objects* and no *Robots*. Volunteers should return *Scoring Objects* to the *Field* as quickly as possible, and any delay in returning a *Scoring Object* should not be considered *Match Affecting* or cause for a replay.

a. A *Scoring Object* that comes to rest on top of the *Field Perimeter* is considered to have left the *Field*.

Interpretation: Our robot may not intentionally remove scoring objects from the field during any part of a match. A scoring object is considered to have left the field if it is completely outside the field or resting on top of the field perimeter.

Impact: We will practice controlling our robot's mechanisms and ensure during the design process that no mechanism uses excessive power that could cause scoring objects to be unintentionally launched out of the field.

<SG5> Each Robot gets one Pin as a Preload. Red *Alliance Preloads* are red/yellow *Pins*; blue *Alliance Preloads* are blue/yellow *Pins*. Prior to the start of each *Match*, each *Preload* must be placed such that it meets all of the following criteria:

- Contacting one *Robot* of the same *Alliance* color as the *Preload*.
- Not contacting the same *Preload* as another *Robot*.
- Not contacting other *Scoring Objects*.
- Not contacting any other *Goals*, *Loaders*, *Load Zones*, or *Toggles*.

Note: If a Robot is not present for their Match, then that Robot's Preload may be used as a Match Load in accordance with <SG11>.

Interpretation: Our robot is allowed one Pin as a preload. To be considered a legal preload, the Pin must be contacting our robot or our alliance partner's robot. It cannot contact another preload, any other scoring objects, Goals, Loaders, Load Zones, or Targets. If the opposing alliance does not show up, the preload may be used as a Match Load.

Impact: We will design our robot to securely hold one Pin as a preload while ensuring it does not contact any prohibited objects on the field. If our alliance partner does not show up, we will use the preload as a Match Load.

<SG6> Possession is limited to a maximum of one Pin and one Cup. Robots may not have *Possession* of more than one (1) *Pin* at once. Robots may not have *Possession* of more than (1) *Cup* at once. Robots in *Violation* of this rule must immediately stop all actions except for attempting to remove the excess *Scoring Objects*.

If they are unable to remove the excess *Scoring Objects*, then they must return to a legal starting position (as described by <SG1>). They will not be eligible to receive points for ending the *Match* in the *Midfield*, and cannot interact with *Toggles*, *Goals*, or other *Scoring Objects* while in *Possession* of excess *Scoring Objects*.

- Plowing* multiple *Scoring Objects* is permitted. Teams which employ *Plowing* strategies are encouraged to clearly demonstrate that none of the *Scoring Objects* are being *Possessed*, e.g., by using a flat face of the *Robot* with no active mechanisms.

Interpretation: Our robot may possess a maximum of one Pin and one Cup at a time throughout the match. The robot may plow multiple scoring objects but cannot intentionally hold more than the allowed possession limit.

Impact: When designing our robot, we will ensure that it can efficiently hold, load, and pick up one Pin and one Cup at a time without exceeding the possession limit. We must also ensure that any surface we'll use for plowing has no concavity, so we do not exceed the possession limit.

<SG8> Engage with the Midfield and/or Autonomous Line during the Autonomous Period at your own risk.

Any *Robot* that engages with the *Midfield* and/or *Scoring Objects* that begin the *Match* on the *Autonomous Line* should be aware that opponent *Robots* may also choose to do the same. Per <GG12> and <GG13>, *Teams* are responsible for the actions of their *Robots* at all times.

- a. For the purposes of this rule, "engages with" means any combination of:
 - i. Contacting foam tiles within the *Midfield*
 - ii. Contacting the *Goal* in the *Midfield*
 - iii. Contacting *Scoring Objects* that begin the *Match* on the *Autonomous Line*

Interpretation: Our robot may interact with midfield scoring objects or objects located on the Autonomous Line during the Autonomous Period. However, doing so increases the risk of crossing onto the opposing alliance's side and receiving a violation.

Impact: Because reaching the Autonomous Line will likely be important for scoring autonomous points, we will repeatedly test and tune our autonomous routine to ensure the robot can interact effectively with midfield objects without crossing into the opposing alliance's side. During the autonomous period, we should also keep an eye on the opposing alliance's autonomous routines, as if they cross the line, we can receive the bonus.

<SG9> Alliance Goals are protected. *Robots* may not directly or indirectly interact with opposing *Alliance*-colored *Goals*. Examples of interactions include, but are not limited to:

- Making contact with the *Goal* or with *Scoring Objects* stacked on the *Goal*. (Direct)
- Adding or removing *Placed Scoring Objects*. (Direct)
- Strategically covering the *Goal* in a manner that prevents additional *Scoring Objects* from being *Placed*. (Indirect)
- Pushing an opposing *Robot* into *Scoring Objects* stacked in their own *Alliance*-colored *Goal*, knocking some or all of them out of the *Goal*. (Indirect)

Interpretation: Our robot may not interact with the opposing alliance's colored Goals at any point during the match, either directly or indirectly.

Impact: This rule primarily applies to our driver; hence, during practice and scrimmages, we will reinforce avoiding any interaction with the opposing alliance's Goals. This rule does provide an advantage and disadvantage for us though. While we cannot contact the opposing alliance goals, they cannot contact ours either. So, we can safely stack as many cups and pins as possible without fear of them being tampered with.

<SG10> Placed Scoring Objects may not be removed from neutral Goals via direct or indirect interactions. Examples of interactions include, but are not limited to:

- Grabbing and lifting a *Scoring Object* out of the *Goal*. (Direct)
- Contacting *Scoring Objects* stacked in the *Goal*, knocking some or all of them out of the *Goal*. (Direct)
- Pushing an opposing *Robot* into *Scoring Objects* stacked in the *Goal*, knocking some or all of them out of the *Goal*. (Indirect)

Interpretation: Placed scoring objects may not be removed from Neutral Goals at any point during the match, whether through direct or indirect interaction.

Impact: This rule primarily applies to our driver; hence, during practice and scrimmages, we will reinforce avoiding any interaction with the neutral Goals. We will also need to be careful about contact with opposing robots while in the vicinity of the neutral goals, as if we push them into a goal and knock it over, that would be a violation.

<SG11> Match Loads may be introduced during the Match under certain conditions. For the purpose of this rule, "introduce" refers to the moment when a *Drive Team Member* has released a *Scoring Object* into a *Loader*.

During this action, a *Drive Team Member's* hands may temporarily break the plane of the *Field Perimeter*. This momentary interaction is an exception to rule <GG4>. Excessive, unnecessary, or unsafe actions while introducing a *Match Load* may be considered a *Violation* of <S1> and/or <G1> at the *Head Referee's* discretion. *Drive Team Members* may introduce *Match Load Scoring Objects* by placing a single *Pin*, a single *Cup*, or a nested *Cup* and *Pin* into either of their *Alliance-colored Loaders*. *Scoring Objects* can be introduced through the top of the *Loader* when the *Loader* is not lifted, or through the back of the *Loader* when the *Loader* is raised by a *Drive Team Member*.

Interpretation: Drive team members may place their hands inside the field perimeter during a match only for the purpose of match loading for their alliance.

Impact: We will ensure that our drive team only places their hands inside the field perimeter while match loading. All drive team members will understand that entering the field perimeter for any other purpose may result in a violation.

<SG12> Some rules change during the Endgame period.

1. *Robots that are at least partially within the Midfield during the Endgame period must make an effort to limit vertical expansion to the approximate starting height of the Robot. If a Head Referee sees a Robot within the Midfield that is clearly expanded past its approximate starting height, that Robot should receive a verbal warning and must either return to approximate starting height or exit the Midfield.*

Version 1.1 - Copyright 2026, Innovation First, Inc./VEX Robotics, Inc.

25

2. *Robots that attempt to end the Match in the Midfield should expect vigorous interactions from opposing Robots. When a Robot is contacting or engaging with the Midfield, or is in proximity to the Midfield, incidental damage that is caused by opposing Robots pushing, tipping, or becoming Entangled with them would not be considered a Violation of <GG14>. Intentional damage or dangerous mechanisms may still be considered a Violation of <S1> or <G1> at the Head Referee's discretion.*

Interpretation: Towards the end of a match during the Endgame, our robot must stay near its starting height while in the midfield.

Impact: We will practice lowering our robot before entering the midfield and design it to withstand heavy contact during the Endgame. Additionally, due to the large amount of pushing required, we will ensure that our robot is able to have high torque and pushing power.

<SG13> Load Zones are protected during the Driver Controlled Period of the Match.

- a. *A Robot is protected while it is at least partially within one of its Alliance Load Zones. Robots may not directly or indirectly contact a protected opposing Alliance Robot.*
- b. *Robots may only enter an opposing Alliance Load Zone momentarily (e.g., while passing through or to retrieve a Scoring Object); Robots may not remain within an opposing Alliance Load Zone.*
- c. *Robots may not cause Scoring Objects to accumulate within an opposing Alliance Load Zone.*

Interpretation: During the Driver Controlled Period, each alliance's Load Zone is protected. Robots may only momentarily enter the opposing alliance's Load Zone and cannot intentionally fill it with Scoring Objects. Robots are also protected while partially within their own Load Zone.

Impact: We will practice ensuring our driver does not remain in an opposing alliance's Load Zone or intentionally fill it with Scoring Objects. We will also remember that our robot and alliance partner are protected while partially within our Load Zones.



08/08/26

Robot Skills Challenge Rules Analysis

Written By: Helena

Proofread By: Trevor

Proofread On: 08/08/26

Goal: Analyze all Robot Skills Challenge Rules in the [Override Game Manual](#) to cement our understanding of the rules and identify any key impacts the rules may have.

<RSC1> Standard rules apply in most cases. All rules from previous sections apply to *Robot Skills Matches*, unless otherwise specified in this section.

- a. Removing *Scoring Objects* from the *Field* in a *Robot Skills Match* is not a *Violation*. *Scoring Objects* that leave the *Field* cannot be returned.
- b. In the Robot Skills Challenge, the standard definition of *Match Affecting* does not apply, because there is no winner or loser. When evaluating whether a rule *Violation* should be classified as a *Major Violation* or *Minor Violation* in the context of this criteria, the term "score affecting" can be substituted for "*Match Affecting*." A *Violation* is considered "score affecting" if it results in a net increase of that *Team's* score at the end of the *Match*.
- c. *Violations* of <GG>, <SG>, and <RSC> rules that occur during a *Robot Skills Match* should only affect the outcome of that *Match* and should not be considered when determining whether a *Violation* has been repeated during the event.

Interpretation: Rules from Robot Game Matches also apply to Robot Skills Matches, except that removing *Scoring Objects* from the field is allowed and Match-Affecting Violations do not apply.

Impact: We will focus more on speed and precision during Skills Matches while recognizing that Match-Affecting Violations from regular matches do not apply.

<RSC2> Match play is different in Robot Skills Matches.

- a. The *Robot* must start the *Robot Skills Match* in a legal starting position in the *Quadrant* adjacent to the red *Alliance Station*.
- b. All *Drive Team Members* must remain in the red *Alliance Station* for the duration of the *Match*.
- c. One red/yellow *Pin* must be used as a *Preload* in accordance with <SG5>.
- d. *Teams* may introduce *Match Load Scoring Objects* at any time during a *Robot Skills Match*, but only through red *Alliance Loaders*.
- e. *Robots* can add or remove *Pins* or *Cups* on all *Goals*.
- f. *Robots* may move freely about the *Field* after the start of the *Match*.

Interpretation: Robot starting position and match play differ from a regular match. The robot must start in the quadrant adjacent to the red alliance station, drive team members may not move around the field, one red/yellow pin must be used as a preload, and robots may add or remove pins and cups from all goals.

Impact: We will ensure our drive team members understand these specific skills rules and practice following them during skills runs in our lab.

<RSC3> Scoring Robot Skills Matches. For each *Robot Skills Match*, *Teams* are awarded a score based on the following rules and scoring table:

- a. *Teams* can earn points for *Pins* of any color.
 - i. Red *Pins* can only be *Scored* if they are *Placed* in a red *Quadrant* or in the *Midfield*.
 - ii. Blue *Pins* can only be *Scored* if they are *Placed* in a blue *Quadrant* or in the *Midfield*.
 - iii. Yellow *Pins* can only be *Scored* if they are *Owned*.
 - 1. *Toggles* only confer *Ownership* of *Pins* when the *Toggle* matches the color of its *Quadrant* (i.e., yellow *Pins* *Placed* in a red *Quadrant* with a *Toggle* that is set to blue are not *Owned* and will not earn any points.
 - 2. Yellow *Pins* *Placed* in the *Midfield* are *Owned* if a *Robot* ends the *Match* in the *Midfield*. (See <SC6>).
 - iv. A *Pin* in contact with a *Robot* at the end of the *Match* does not score points.
- b. The *Team* will earn points for a *Robot* ending in the *Midfield*. (See <SC6>.)

Interpretation: The robot may score pins of any color. Red and blue pins only score in their matching quadrant or midfield, while yellow pins must be owned to score. The robot must end in midfield for additional points, and pins contacting the robot at the end of the match are not scored.

Impact: Knowing these rules, we will build our skills path to maximize scoring opportunities while remaining within the scoring requirements. We will also leave enough time to end in midfield when the additional points are worthwhile.

<RSC4> Field setup for Skills Matches. The *Field* is set up differently than a *Head-to-Head Match*, with the following modifications:

- a. In *Autonomous Coding Skills Matches*, the VEX GPS code strip must be installed on the *Field*.
- b. Revised *Match Loads*. Three (3) red/yellow *Pins*, four (4) blue/yellow *Pins*, and seven (7) *Cups* begin off the *Field* as *Match Loads*.
- c. All *Goals* start the *Match* empty with no *Placed Pins*.

Interpretation: Our robot will compete on a different field setup than a regular match. All goals will start empty, with 3 red/yellow pins, 4 blue/yellow pins, and 7 cups available as match loads.

Impact: We will plan our skills strategy around starting with the empty goals and efficiently using the available match loads to maximize scoring throughout the run.

<RSC5> Skills Stop Time. If a *Team* wishes to end their *Robot Skills Match* early, they may elect to record a *Skills Stop Time*. This is used as a tiebreaker for Robot Skills Challenge rankings. A *Skills Stop Time* does not affect a *Team's* score for a given *Robot Skills Match*. *Drive Team Members* and field staff must agree prior to the *Match* on the signal that will be used to end the *Match* early.

If the event is utilizing a V5 Robot Brain or the TM Mobile app for Robot Skills Challenge field control, a *Drive Team Member* may elect to start and stop their own *Robot Skills Matches*.

Interpretation: If teams wish to end a skills match early, drive team members must agree and notify field staff before the skills run begins. The ending time will only be used as a tiebreaker between teams with equal scores and will not affect the skills score itself.

Impact: Because our team will use a V5 Robot Brain, a drive team member will start and stop our skills run. We will only attempt an early stop time if we have practiced it consistently and determined it improves our placement.



08/09/26

Other Important Rules Analysis

Written By: Helena

Proofread By: Trevor

Proofread On: 08/10/26

Goal: Analyze any rules outside of the 3 game specific categories that we deem especially important, or have changed from last year in the [Override Game Manual](#).

<R26> The VEX Pressure Gauge (276-8748) is a required part if a Robot includes pneumatics.

- a. The gauge must be plumbed to the high pressure side of the system if the *Team* is using a regulator.
- b. The pressure gauge must be visible and readable by referees and inspectors without moving or removing other *Robot* components or mechanisms.
- c. The pressure gauge will be used to verify that the pneumatics system is charged to no more than 100psi, per <R25>.
 - i. The VEX Pressure Gauge on the *Robot* will take precedence over any off-board measurement.

Interpretation: A robot that uses pneumatics must include a VEX pressure gauge that is visible to inspectors. The gauge must show that the pneumatic system does not exceed 100 psi.

Impact: This is important because the pressure gauge is a brand new requirement this year. If we use pneumatics, we will need to incorporate the gauge into our design and CAD from the beginning to make sure it remains visible and accessible in order for us to pass inspection.

<R11> Motors are limited for Subsystem 2 (see <R1>). Robots may use any combination of V5 Smart Motors, 11W (276-4840) and/or 5.5W (276-4842), in Subsystem 2 within the following criteria:

- a. The combined power of all motors in Subsystem 2 must not exceed 55W.

Interpretation: Robots are not permitted to exceed 55W for the drivetrain.

Impact: This rule is important for our team because it directly affects our build process and drivetrain design. Last year, we used a 66W drivetrain, but this year we will be limited to 55W. This may reduce drivetrain power and acceleration, so we will account for this in our gearing and overall design while staying within the rules.



08/09/26

Key Takeaways/Initial Strategy

Written By: Sam

Proofread By: Trevor

Proofread On: 08/10/26

Goal: Compile our key takeaways from our analysis of the [Override Game Manual](#) to identify the central problems our robot needs to address; then, create an initial strategy.

Qualification/Elimination Matches

In the past years, games with possession limits favored faster robots. For example, Over Under and High Stakes both favored fast robots that can swiftly pick up and score objects, reducing their “cycle time”. Similarly, with a limit of 1 cup and 1 pin, we believe that faster robots will be beneficial, which is why we will likely implement a 55 watt drivetrain, which will decrease our cycle time for picking up objects and scoring them on the goal. We will also utilize the match loader often, as it consistently shortens the distance between objects and the goals.

For match strategy, the object that gives us the greatest potential for points is the toggles. Our goal is to control and defend the toggles on our side while scoring yellow pins. Also, scoring as many of our colored pins on the opposing side's neutral goals will limit the number of pins that could count towards our opponents.

Skills Challenge

For skills, our goal is to score as many yellow pins as possible and own them with the toggle, as that gives the greatest point potential. We will also end in the midfield for the bonus 8 points.

Conclusion/Initial Strategy

In conclusion, we found that fast driving, as well as controlling the yellow pin value, will give us the greatest potential for victory. In our first tournament, we will prioritize controlling the toggles, scoring as many yellow pins as possible, and trying to score our color on the opponent's neutral goals.

Robot 1

“Frostbyte”

Time Management: Robot 1

Goal: Create a Gantt Chart and outline our process for our robot prior to our first competition.

[illegible]

Above is the Gantt Chart for the timeline we wish to follow. Please note that this is a **rough guideline** we will be following, not an exact pace to follow. We are still very early in the season, so we know unexpected issues may arise when designing, building, and programming our new robot. We've **attempted to add buffer room** in the schedule, but we of course cannot predict the future so this is just a guide for now. We will update the Gantt Chart as needed and create weekly Gantt Charts in our weekly update entries to reflect real progress.

Important Timing Notes

This past season, our old coach left and we received a new one. Because of this, there have been a few scheduling issues and as of right now we don't know when our first competition will be. Our estimate is that our first competition will be either **late October or early November**. Because of this, we aim to have our robot (including programming) finished by then. Once we learn our full tournament schedule, we will update our Gantt Charts to reflect that.

You'll notice that there is a large portion of time dedicated to programming. We have this time in there because over the past few years of competing, **we've realized how key the autonomous period is**. Not only does the period provide the autonomous bonus, but it also provides the autonomous win point (which helps us rank higher) and provides a strategic advantage for the start of the driver control period. Hence, we've allotted a significant portion of our time towards writing and tuning efficient, high scoring, and consistent autonomous routines.



08/05/26

Code Initial Setup: EZ Template

Written By: Trevor

Proofread By: Jasper

Proofread On: 08/05/26

Getting Started with EZ-Template

To begin setting up our code, we navigated to [EZ-Template's Installation page](#), where we downloaded the latest version. Next, we opened the folder using our preferred text editor for coding, Visual Studio Code (VS Code). We prefer VS Code over other text editors because it is fairly simple to set up, has many extensions that help with overall quality of life when coding, and works seamlessly with Git/Github.

Setting up Version Control with Git/GitHub

Before we explain how we set up version control, we want to quickly explain what Git and GitHub actually are.

Git is a version control system that tracks the history of files in a given repository, which is essentially just a folder that contains all your code files. This repository, however, is stored locally on the system from which it was created. To publish this repository to the cloud so that other people can collaborate on the repository, you must use a Git server such as GitHub. There are other Git servers that exist other than GitHub, but GitHub is the most common. Once published to GitHub, other people, in this case the rest of the team, can open the repository and make their own changes.

The simplest way we've found to create a repository is through the GitHub Desktop App. Once the application is open, we simply select the "New repository" option, select the folder that we wanted to make a repository of (i.e., the folder that contained our code), give it a title, and then publish it to GitHub. We have not made our repository yet, however when we do, we will add both a QR code and a link in [Appendix A](#) at the end of the notebook.

Now that the repository is created, we can now commit changes to the code through the "Source Control" feature of VS Code, and the rest of the team will be able to see the changes.

*** Note: To actually be able to open and use a repository in a text editor (i.e., VS Code), you must have Git installed and properly set up on your device.**

Code Formatting/Organization

The last thing I did today was create ".cpp" and ".hpp" for each subsystem (e.g., intake, match loading mechanism) that we have or are planning to have. We could have all the code for all our subsystems in the same file, but having them separate makes it much easier to navigate.



08/05/26

Identify: Drivetrain

Written By: Trevor

Proofread By: Helena

Proofread On: 08/05/26

Goal: Using our knowledge from the Override Manual, outline the design constraints and goals for our drivetrain.

Problem Statement

We need to design a mechanism that will not only allow our robot to move around the field, but also contribute to our strategic goals.

Constraints

- Must use only VEX legal parts and be no larger than 18" x 18".
 - This is an inherent constraint set by the game manual.
- Must use a maximum of 55w worth of motors.
 - This constraint is set by the new drivetrain limit rule in the game manual.
- Our driver (Sam) must be able to comfortably operate it.
 - While some designs may provide benefits in theory, if the person operating the drivetrain cannot effectively control it, the benefits become almost completely nullified.

Design Goals

- Must have good acceleration, but also overall speed.
 - This is important because we want to quickly get from place to place, whether for short or long distances.
- Must be able to effectively and quickly change directions.
 - We want to react to sudden changes in the game, and being able to change where we are driving to in an instant will help.
- Must be very precise while moving.
 - There are many obstacles on the field due to the base of the towers. Hence, the drivetrain must allow extreme precision to navigate between the bases.



08/06/26

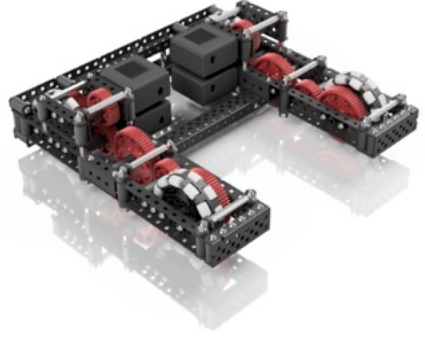
Research: Drivetrain Types

Written By: Jasper

Proofread By: Trevor

Proofread On: 08/07/26

Goal: Research common drivetrains from past seasons to help determine the optimal type for this season.

Tank Drive		
Description	A common VEX chassis design that utilizes an even number of wheels distributed on each side. Its primary movement is limited to forward, backward, and turning. Tank drives, when equipped with certain wheels, can resist sideways motion or slip when pushed. They are highly versatile for motor consumption, accommodating various configurations of 11W or 5.5W motors.	
Pros	● Excellent pushing force in forward and reverse. ● Minimal frictional losses. ● Generally compact, easy to build, and straightforward to program. ● Often lightweight. ● Good at traversing obstacles. ● Sam has lots of experience driving them. ● Easy to drive	 Source: VEX Forum
Cons	● Typically 4 or 6 motors. Would have to adapt to the 55W limit ● No lateral movement ● Requires many supports ● Doesn't allow for strafing	

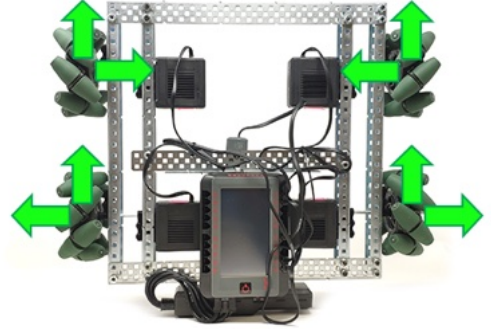
H-Drive

Description	A type of holonomic drive that uses five Omni-wheels arranged in an "H" pattern. Each wheel is normally directly linked to a motor; however, each side of the drive can also be linked together. This chassis can strafe by using the single wheel perpendicular to the others to push itself sideways. Despite having the perpendicular wheel, H-drives generally offer little resistance to pushing.	
Pros	● Inherently 5 motors ● Very sturdy ● More compact than other strafing designs. ● Offers true omnidirectional movement.	 <i>Source: Levitan</i>
Cons	● Requires an extra motor ● Takes up more space in the middle ● No traction ● Inconsistent strafing with weak power. ● The perpendicular wheel can lose traction when there are field imperfections. ● Awkward motor configuration. ● Programming its omnidirectional movement can be complex.	

X-Drive

Description	A type of holonomic drive consisting of four Omni-wheels arranged in an "X" pattern, with each wheel independently powered by a motor. By adjusting the wheels' speeds in different combinations, the robot can move forward, backward, sideways, or rotate in place. With this ability, movement can be more effectively used than with other drivetrains.	
Pros	● Typically 4 motors ● Fast turning ● Lots of movement vectors ● High maneuverability	 <i>Source: VEX Forum</i>
Cons	● Hard to drive ● Not a lot of traction ● Lower power ● Much harder to use 55W ● Little pushing resistance. ● Hard to add more mechanisms.	

Mecanum Drive

Description	A type of drive that uses four mecanum wheels with several rollers mounted at a 45° angle to move around the field. These four wheels are mounted similarly to a tank drive; however, the wheel gap must be much wider to fit the thicker mecanum wheels. The robot moves forward, backward, and turns similarly to a tank drive. This drive also moves similarly to an X-Drive (e.g., strafing, turning in place) but is usually slower and less efficient.	
Pros	• Typically 4 motors • Very useful movement, allowing high maneuverability • Allows for more advanced programming routes	 Source: CS-STEM
Cons	• Hard to drive • Not a lot of traction • Often overkill movement • Cannot connect wheels together: less power • Large frictional losses when turning and strafing. • Bulky wheels. • Slower overall speed. • Reduced pushing power in strafing directions.	

After this research, we have decided not to consider a mecanum drive as an option in the future. It has many more disadvantages than advantages, and removing it as a contender will allow us to research and evaluate only 3 designs, which will be much more time efficient.



08/07/26

Research: Wheel Sizes & RPM

Written By: Trevor

Proofread By: Jasper

Proofread On: 08/08/26

Goal: Research available wheel sizes and RPM configurations to determine the optimal setup for Robot 1.

Wheel Size and RPM Options

In VEX V5RC, there are currently three main wheel size options available (Source: [VEX Website](#)): **2.75"**, **3.25"**, and **4" diameter**. There is also a 2" size available for Omni-wheels, but the omnidirectional rollers have gaps between them that cause issues during programming and driving.

Additionally, three main RPM configurations are used: **360 (36:60)**, **450 (36:48)**, and **600 (36:36) RPM**. The numbers in the parentheses are the respective gear ratios (Input Gear Teeth: Output Gear Teeth) we would use to achieve these RPMs. We could consider other gear ratios and RPMs, but they are either too fast, too slow, or have too complex gearing to be a practical option.

Speed Calculation Process

Tank Drive/H-Drive:

$$Circumference_{wheel} = Diameter_{wheel} \cdot \pi$$

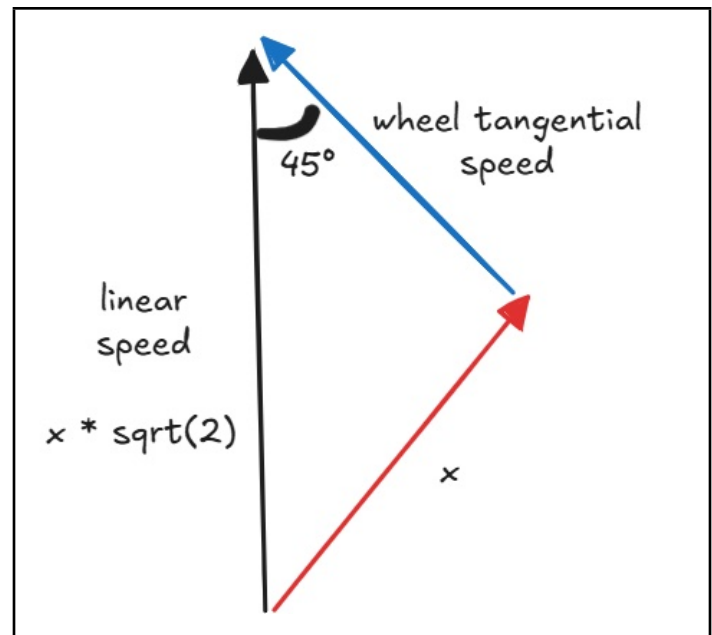
$$Speed = Circumference_{wheel} \cdot RPM$$

X-Drive:

$$Circumference_{wheel} = Diameter_{wheel} \cdot \pi$$

$$Speed_{tangential} = Circumference_{wheel} \cdot RPM$$

$$Speed = Speed_{tangential} \cdot \sqrt{2}$$



X-Drive Explanation: An X-Drive's wheels are angled at 45°, meaning the robot's linear speed is the vector sum of each wheel's tangential velocity. Because the tangential velocities create a 45-45-90 triangle, we know that the linear speed is $\sqrt{2}$ times faster than the tangential speed of a single wheel.

Acceleration Calculation Process

These calculations assume that only four motors are being used (for H-Drive, the perpendicular motor does not contribute to acceleration), that the stall torque of a 600 rpm motor is 0.35 Newton meters (Nm), and the drivetrain weight is 4 pounds (1.81 kg).

Tank Drive/H-Drive:

$$Radius_{wheel} = \frac{Circumference_{wheel}}{2}$$

$$Torque_{per_wheel} = 0.35 \text{ Nm} \cdot \frac{Output \text{ Gear Teeth}}{Input \text{ Gear Teeth}}$$

$$Linear \text{ Force}_{per_wheel} = \frac{Torque_{per_wheel}}{Radius_{wheel}}$$

$$Linear \text{ Force}_{total} = Linear \text{ Force}_{per_wheel} \cdot 4$$

$$Acceleration = \frac{Linear \text{ Force}_{total}}{1.81 \text{ kg}}$$

X-Drive:

$$Radius_{wheel} = \frac{Circumference_{wheel}}{2}$$

$$Torque_{per_wheel} = 0.35 \text{ Nm} \cdot \frac{Output \text{ Gear Teeth}}{Input \text{ Gear Teeth}}$$

$$Linear \text{ Force}_{per_wheel} = \frac{Torque_{per_wheel}}{Radius_{wheel}}$$

$$Linear \text{ Force}_{total} = \frac{Linear \text{ Force}_{per_wheel} \cdot 4}{\sqrt{2}}$$

$$Acceleration = \frac{Linear \text{ Force}_{total}}{1.81 \text{ kg}}$$

*** All these calculations (speed and acceleration) are theoretical and assume ideal conditions, as we just need a rough comparison of the configurations. Additionally, unit conversions are not included in these steps but were considered when calculating the results.**

Speed Results

Tank Drive/H-Drive	2.75"	3.25"	4"
360 RPM	51.83 in/s	61.22 in/s	75.40 in/s
450 RPM	64.80 in/s	76.52 in/s	94.25 in/s
600 RPM	86.40 in/s	102.10 in/s	125.67 in/s

X-Drive	2.75"	3.25"	4"
360 RPM	73.30 in/s	86.57 in/s	106.63 in/s
450 RPM	91.63 in/s	108.20 in/s	133.33 in/s
600 RPM	122.18 in/s	144.38 in/s	177.72 in/s

From these two tables, we can see that for all three types of drivetrains, 360 RPM on 2.75" wheels is the slowest configuration, while 600 RPM on 4" wheels is the fastest configuration. Additionally, 450 RPM on 3.25" wheels is the median of both tables, meaning that it provides a moderate speed in comparison to the others.

Acceleration Results

Tank Drive/H-Drive	2.75"	3.25"	4"	X-Drive	2.75"	3.25"	4"
360 RPM	1453.5 in/s ²	1230.0 in/s ²	999.2 in/s ²	360 RPM	1028.0 in/s ²	870.0 in/s ²	706.7 in/s ²
450 RPM	1162.2 in/s ²	983.5 in/s ²	799.2 in/s ²	450 RPM	822.0 in/s ²	695.3 in/s ²	565.0 in/s ²
600 RPM	872.0 in/s ²	737.8 in/s ²	599.6 in/s ²	600 RPM	616.5 in/s ²	521.7 in/s ²	424.0 in/s ²

From the results, we can see that for all three types of drivetrains, 600 RPM on 4" wheels has the least acceleration, while 360 RPM on 2.75" wheels has the most acceleration. Similar to the speed results, 450 RPM on 3.25" wheels is the median of both tables, meaning that it provides a moderate acceleration in comparison to the other configurations.

Conclusion

After analyzing this data, we have concluded that **450 RPM on 3.25" wheels is the best option for all three types of drivetrains** (Tank Drive, H-Drive, X-Drive). We came to this conclusion as this configuration provides a good balance between maximum speed and acceleration (as shown by the fact it was the median of both results). Additionally, 3.25" wheels also provide excellent traversal over minor field inconsistencies and obstacles, such as the Park Zone barrier.

360 RPM on 4" wheels was another extremely viable alternative, as it was very close to 450 RPM on 3.25" wheels in all of the results and was the closest to the mean of each table. Ultimately, the fact that 4" raise the center of gravity was a dealbreaker, as 50 inch vertical expansion requires a low center of gravity to prevent tipping. However, in the future, if we run into issues with 450 RPM on 3.25" wheels, our backup option will almost certainly be 360 RPM on 4" wheels.



08/07/26

Research: Wheel Options

Written By: Helena

Proofread By: Trevor

Proofread On: 08/07/26

Goal: Research available wheel types to determine the optimal setup for Robot 1.

Omni-wheels		
Description	Omni wheels are wheels with small rollers around the outside that allow the robot to move forward and backward normally while also sliding sideways with very little friction.	
Pros	• Omnidirectional movement. • Smoother turning and overall driving. • Improves driver maneuverability.	 Figure Source: VEX Website
Cons	• Reduced linear pushing power due to reduced grip on tiles. • Easier to be pushed around sideways. • Can drift sideways during autonomous routines. • Slightly worse obstacle traversal.	

Traction Wheels		
Description	Traction wheels are wheels with a strong rubber tread around the outside, which provides more grip and helps prevent the robot from slipping or being pushed by other robots.	
Pros	• Improves linear pushing power due to increased traction. • Easier obstacle traversal. • Helps resist sideways pushing.	 Figure Source: VEX Website
Cons	• Significantly reduces driver maneuverability. • Can make coding more complex autonomous movements such as curves harder.	

*Note: Flex wheels are not considered here, as they are not as effective when used as drivetrain wheels due to their flexible nature. They will, however, be considered when brainstorming other subsystems.

Conclusion

We have concluded that using 4 omni-wheels and 2 traction wheels for a tank drive would be the best fit for our team. On each side, we will have 2 omni-wheels and one traction wheel. We believe this setup will allow us the most maneuverability, while also maintaining traction. Additionally, Sam used this setup last year when he was on 12Z, so he has experience with it.



08/08/26

Research: MoA Robots

Written By: Jasper

Proofread By: Trevor

Proofread On: 08/09/26

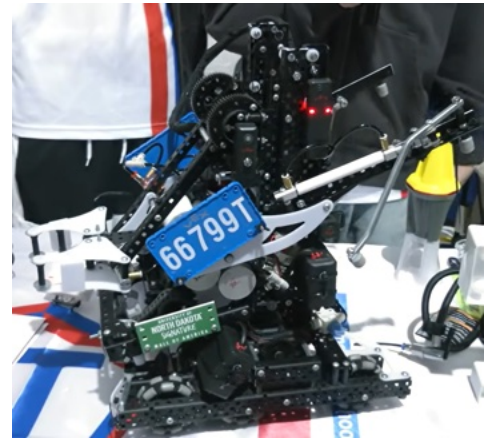
Goal: Identify and evaluate high performing and unique robots from the first signature event, Mall of America, to learn about key strengths and weaknesses.

Team 66799T

Description A robot with a continuous lift and claw arm; uses an intake to guide cups/pins to the back of the robot; matchloads via claw; toggles rollers using a pneumatic powered side lever.

**Promising/
Interesting
Ideas**

- The claw using matchloading in front and intake loading in back seems concise and elegant
- The pneumatic powered side toggler offers a useful way to grab toggles at the end of the match
- Continuous lift is compact, fast, and reliable



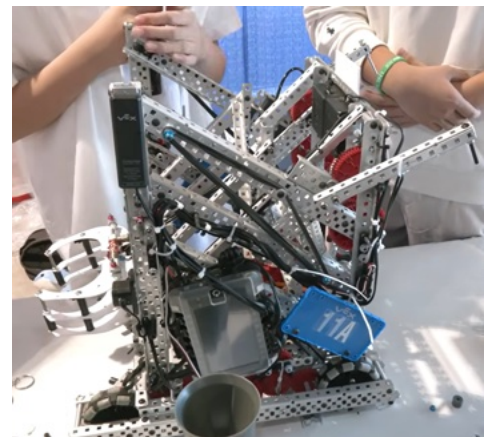
Source: 66799T

Team 11A

Description Our number neighbor, a highly effective DR4B with a claw that utilizes its quick lift to create high stacks at a rapid pace; uses a similar toggle mechanism to 66799T; very fast drivetrain allows for all-around quick scoring on all field quadrants

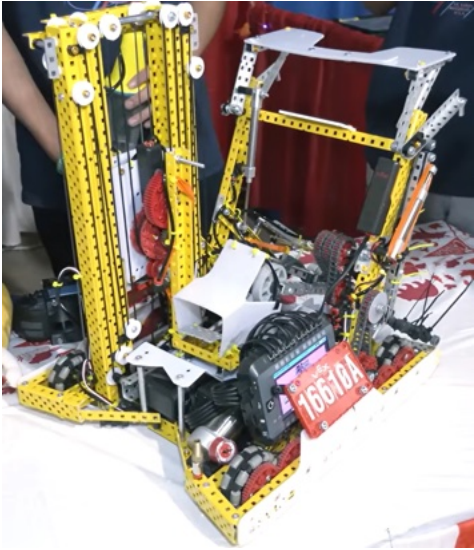
**Promising/
Interesting
Ideas**

- Double reverse four bar (DR4B) lift is sturdy and easy to build
- Claw seems like the best pneumatic way to grab pins/cup
- Matchload pin: has a pin sit on the intake for autonomous routines (high consistency)



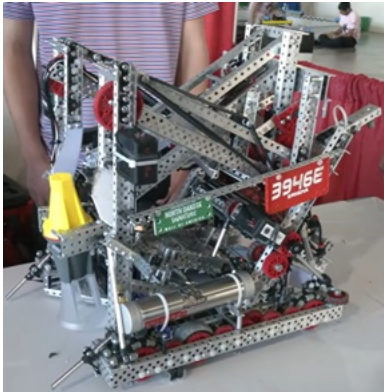
Source: 11A

Team 16610A

Description	Team 16610A uses a differential lift, a passive plastic claw, passive toggles, a pistonized intake (pneumatically able to grab matchloads), and a conveyor belt intake.	
Promising/Interesting Ideas	• A differential lift allows for two motors to serve two purposes, yet each purpose with 22W. For example, motors spinning the same direction controls the lift; motors spinning opposite directions turn the passive claw. Very useful. • An aligner to the matchload tube • Using pistonized intake, we can avoid a bit of trouble in constructing a new matchload mechanism • The conveyor belt intake works well and seems reliable	

Source: 16610A

Team 3946E

Description	Team 3946E uses impressive sensor technology, a DR4B, a pneumatic claw, wedges, a front-stage intake and interesting banding to get the lift up.	
Promising/Interesting Ideas	• Triangle banding is utilized for a faster lift • A distance sensor on the claw is used for autoclamping • A light-weight lift helps speed as well • The claw is pneumatic without the use of plastic • Distance sensors and odometry are used for high accurate autonomous routines	

Source: 3946E

Conclusion

Based off of the robots we evaluated at the Mall of America Signature Event, we would like to include:

- A reliable, concise way to raise/lower game objects
- A claw to grab game objects
- A smooth intake to pick objects off the field
- A simple matchloading mechanism
- A way to toggle rollers passively
- Ease of scoring low game objects for autonomous routines



08/09/26

Research: MoA Analysis

Written By: Jasper

Proofread By: Trevor

Proofread On: 08/10/26

Goal: After watching the Mall of America Signature event, identify notable designs/mechanisms and strategies.

In order to determine effective strategies and design mechanisms for match play at the most competitive level, we will watch and analyze the Mall of America (MoA) Signature Event. Most of the analysis will be in the elimination rounds, as that is where the highest level of match play exists.

Notable Designs/Mechanisms

Goal Aligners

Put simply, a piece/pieces of plastic shaped to align the robot to the goal. In our viewing, we notice manually aligning to each goal takes time, while teams with an aligner were much faster and far more successful.



Source: 1698Z

Intake Matchloading

Utilizing matchloaders from the front stage intake, as well as from the back stage allows pin/cup pairs to have either orientation. Intake matchloading seems quite fast, as it removes the need to turn around when scoring.



Source: 10102A

Odometry + Consistent Autonomous

Several autonomous routines failed in elimination matches, causing them to be ill setup for their driver control period. Teams who had odometry, however, had more consistent autonomous routines, and therefore performed better in driver control.



Source: 11981C

Passive Toggles

As discussed in MoA robots, the ability to toggle rollers is extremely useful; the passive toggles we noticed were extremely effective and highly utilized throughout the matches



Source: 1028A

Notable Strategies

Matchload Alliance Color Repetition

Using matchloads of alliance color on alliance goals to stack as much as possible.

- Highly effective, can get many points very quickly
- Requires a bot with a quick lift
- Can only be done in one corner of the field
- Heavily utilized by 10102A to ultimately win the tournament

Roller Control

A more defensive strategy: using toggle mechanisms, set each corner of the field to alliance color, especially corners with many yellow pins.

- Doesn't require heavy scoring/offensive capabilities
- High point potential
- Requires constant field monitoring — no payoff if you lose the roller
- Requires a fast robot and high maneuverability

Yellow Pin Play

Teams prioritize the yellow side of the pin in order to get more points.

- Has extremely high point potential
- Dangerous: a single roller flip causes it to be counterproductive
- Harder to utilize matchloads, easier to utilize field elements
- Requires a degree of roller control

Other Insights

Self-knocking the Alliance Goal

Since you're allowed to descore your own alliance stack, teams that lost possession of their toggle and couldn't get it back knocked over their own stack to prevent the other team from getting points from the yellow pins.

Tipping and Anti-Tip

Several teams experienced near-tipping events in matches. Such events are match-ending, so teams take precautions to not tip over, such as detection code or manual hardstops.

King of the Hill Underusage

Due to the importance of toggles at the end of the match, very few teams go for the "King of the Hill" endgame challenge. Being underutilized, perhaps we place a bit less emphasis on it for the early season (unless, in the event, we have adequately secured our roller).



08/10/26

Final Drivetrain Options

Written By: Trevor

Proofread By: Helena

Proofread On: 08/10/26

Goal: Based on the previous entries, outline our final design options before selecting one.

For each drivetrain option, we will outline its key characteristics (e.g., motor usage). However, we will not present a visual representation of each option, as they will not significantly differ from their corresponding pictures in the [Drivetrain Types entry](#) (pg. 40).

Option 1 (X-Drive)

- All Omni-wheels (inherent requirement of the design type)
- Four wheels (simplest and most effective way to build)
- Will use four 11w motors (one motor per wheel)
- Must use a configuration with 450 RPM and 3.75" wheels

Option 2 (H-Drive)

- All Omni-wheels (inherent requirement of the design type)
- Five wheels (four for vertical movement, one for horizontal movement)
- Will use five 11w motors (one motor per wheel)
- Must use a configuration with 450 RPM and 3.75" wheels

Option 3 (Tank Drive)

- Part Omni-wheels, part traction wheels (receives most benefits for maneuverability and stability)
- 4 Omni-wheels + 2 traction wheels (optimal distribution)
- Will use four 11w motors + two 5.5w motors (meets 55w limit while maintaining speed + power)
- Must use a configuration with 450 RPM and 3.75" wheels.

*Note: These options are based on what we've learned from our Research entries.



08/11/26

Evaluate & Select: Drivetrain

Written By: Trevor

Proofread By: Jasper

Proofread On: 08/12/26

Goal: Evaluate our [final drivetrain options](#) using a design matrix so we can begin building.

As we mentioned in the [Design Matrices entry](#) (pg. 9), design matrices allow for comprehensive comparisons and objective evaluations. In order to thoroughly evaluate the advantages and disadvantages that each drivetrain option has, we need to consider how much each advantage/disadvantage matters in relation to our design goals and other criteria that we find important; weighing criteria in a design matrix will allow us to do this. Our primary design goals will be weighted higher than other less important criteria.

Criteria	Description	Weight
Acceleration	We will judge this based on how quickly the drivetrain accelerates. Acceleration data will come from the Acceleration Results on pg. 43.	7
Maximum Speed	We will judge this based on the maximum speed of the drivetrain. Maximum speed data will come from the Speed Results on pg. 42.	6
Barrier Traversal	We will judge this based on how effectively and quickly the drivetrain would theoretically traverse the Park Zone barrier.	6
Change of Direction	We will judge this based on how quickly the drivetrain can change directions (i.e., turn or strafe).	4
Ease of Expansion	We will judge this based on how easy it will be to build additional subsystems on top of the drivetrain.	3
Simplicity	We will judge this based on the amount of components the drivetrain requires and how easy the build process is.	2
Weight	We will judge this on how light the drivetrain would theoretically be. This means that the higher the score, the lighter the drivetrain.	2

We will score the drivetrain options on the following criteria out of 10 based on our best judgment, which will be justified by either our Background Research or knowledge we've gathered from our experience with VEX. If the robot is incapable of completing any of the actions, it will be given a score of 0 in that criteria and it will be noted.

Because most of these criteria are not based on quantitative data, the scores will be relative to the other options.

Design Matrix

Criteria:	Weight	Option 1 (X-Drive)		Option 2 (H-Drive)		Option 3 (Tank Drive)	
		Score	Weighted Score	Score	Weighted Score	Score	Weighted Score
Acceleration	7	5	35	7	49	8	56
Maximum Speed	6	8	48	5	30	6	36
Barrier Traversal	6	3	18	5	30	8	48
Change of Direction	4	9	36	7	28	4	16
Ease of Expansion	3	5	15	7	21	8	24
Simplicity	2	5	10	7	14	9	18
Weight	2	5	10	6	12	7	14
Total Score:			172		184		212

Conclusion

The design matrix has identified Option 3 (Tank Drive) as the best option. Option 3 received a weighted score of 212, with Option 2 and Option 1 receiving lesser scores of 184 and 172, respectively. Thankfully, this drivetrain is not extremely complex and does not require a large amount of resources, so we will not need to request any extra parts from our coach. Our next step will be to select our remaining subsystems before beginning to CAD our robot.



08/12/26

Identify: Remaining Subsystems

Written By: Helena

Proofread By: Jasper

Proofread On: 8/13/26

Goal: Using our knowledge from the [Override Game Manual](#), outline the design constraints and goals for the remaining subsystems of our robot.

Problem Statement

We need to design subsystems that will allow us to score and perform necessary actions with the field elements (e.g., unload Match Loaders).

Constraints

- Must only use legal parts and be no larger than 18"x18"x18" when attached to our drivetrain.
 - This is an inherent constraint set by the game manual.
- Must use a maximum of 88w worth of motors.
 - Only 33w worth of motors allowed for the drivetrain.
 - Only 55w worth of motors allowed for the rest of the robot.

Design Goals

- A mechanism to intake pins and cups
- A lift to raise scoring objects to the correct heights.
- A toggle to change field toggles to our alliance color.
- A claw to control/grab scoring objects.
- A winch to raise and lower our lift.

Optional Design Goals

- Low weight to keep our robot as light and agile as possible.
- Compact design to keep our robot easily maneuverable for our driver (Sam).
- Low friction to allow for smoother movement throughout all of our robot mechanisms.
- Consistency: we are aiming for consistency in our accuracy and performance throughout the season, so we can prioritize improving our design, and decreasing variability so that we are not constantly going back and fixing little issues.



08/24/26

Game Strategy Development

Written By: Sam Kim

Proofread By: Trevor

Proofread On: 08/25/26

Goal: Use our game analysis, initial ideas, and MoA research to develop an early season strategy to guide build, code, and practice time.

The Strategy

Autonomous Period

We will need at least two different kinds of autonomous routes. One for the quarter closest to us, and one for the quarter to the right of us. Since the field isn't mirrored, we can run the same autos for the red and blue side.

Our main two objectives during the autonomous period will be to set up our match load cycles and secure the toggle. In order to do so, we will flip the toggle first, and then place at least one pin on the alliance goal closest to us. Since we mainly hold our stacks with the cup and then the pin on top, starting the match with the pin-side up will help reduce cycle speed. Also, scoring on our own alliance goal is most optimal, so that we have the ability to take down our stack if the possession gets flipped.

Driver Period

During the driver period, our goal is to score as fast as possible while maintaining our toggles. To do so, we plan to have robots switch roles between scoring and offense/defense throughout the match. At the beginning, one robot will place only our colored pins on the quarter to the right of us. This way, it is impossible for those points to get swung back and forth. While this is happening, the other robot will go attack the opposing team's toggle and disrupt the flow of their scoring. We do not need to worry about opening up any toggles for our opponents during that time, as all our points will be from alliance colored pins. Once the first team is done scoring, we start scoring yellow pins on the alliance goal in the quarter closest to us. This way we have the ability to tear down if needed, and it also has the toggle that is closest to us. While this is happening, the other robot comes back to play defense on our toggle, making sure our yellow stack doesn't get flipped.

Endgame

At the end of the game if, and only if we feel that our toggles are under no threat, we will go for the king of the hill in the middle for more points.

Adjusted Design Goals

Our desire to matchload swiftly may change our claw design to grab it straight out of the loader without intaking it



08/13/26

Final Subsystem Design Options

Written By: Woodley

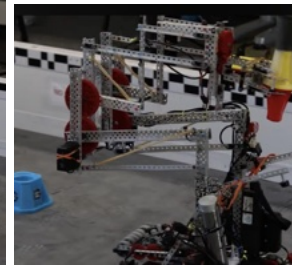
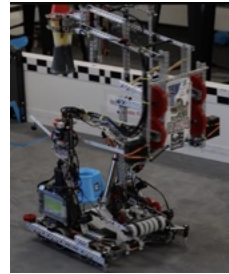
Proofread By: Helena

Proofread On: 8/14/26

Goal: Using the information from our background research and brainstorming, outline the best possible designs for the main body of our robot.

Double Reverse 4-Bar

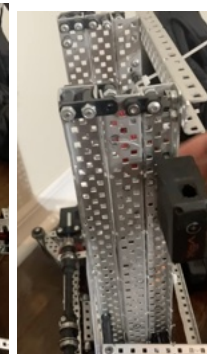
- Extends straight upward vertically
- Reaches higher than a regular four-bar
- Level mechanism
- Very stable
- Heavy due to the amount of structure
- Takes up significant space on the robot



2982B

Cascade

- Uses string, chain, or other rigging
- 2 main types: continuous and cascade rigging; typically use 1 vs. 3 pieces of string
- Can use bearings or screw joints to connect and guide each stage
- Lots of metal-on-metal contact, which indicates possible heavy friction



13722A

6-Bar

- Similar to a four-bar, but provides additional height
- Folds down compactly
- Level mechanism
- Relatively simple to build
- Circular arc (not vertical)
- Takes up significant space



RoboSTEM



08/14/26

Evaluate & Select: Subsystem Design

Written By: Trevor

Proofread By: Jasper

Proofread On: 08/15/26

Goal: Evaluate our [final subsystem design options](#) using a design matrix so we can begin building.

This design matrix will function almost identically to the one we used when we [selected our drivetrain design](#) (pg. 49), but will feature different criteria. These criteria will primarily consist of ways to assess how the options line up with our design goals, but there will also be some generic criteria that we want to consider, but will have a significantly lesser weight.

Criteria	Description	Weight
Lift Speed	We will judge this based on how quickly the robot will be able to fully extend to its maximum height and then return to its base level.	7
Scoring Capability	We will judge this based on the robot's ability to score on each side of the robot (full points for 2 sides, half points for 1 side)	6
Claw/Intake Ability	We will judge this based on how effectively and quickly the intake and/or claw can pick up pins/cups in order to score them.	6
Cycling Efficiency	We will judge this based on how easily the design can go from collecting pins/cups from a Match Loader to scoring them in a Goal.	5
Ease of Expansion	We will judge this based on how easy it will be to build additional subsystems (i.e. mechanisms for toggles and other subsystems).	3
Durability	We will judge this based on how likely the robot is to break during a match. (This is judged primarily on our MoA Robots research (pg. 46)).	3
Simplicity	We will judge this based on how simple the design is to build and repair.	2

We will score the drivetrain options on the following criteria out of 10 based on our best judgment, which will be justified by either our Background Research or knowledge we've gathered from our experience with VEX. If the robot is incapable of completing any of the actions, it will be given a score of 0 in that criteria and it will be noted.

Because we have no prototypes that we can use to acquire quantitative data from, these criteria will be scored relative to each other and based on our best judgement and general knowledge.

Design Matrix

		Option 1 (Double Reverse 4-Bar)		Option 2 (Cascade)		Option 3 (6-Bar)	
Criteria:	Weight	Score	Weighted Score	Score	Weighted Score	Score	Weighted Score
Lift Speed	7	8		9		6	
Scoring Capability	6	5		10		5	
Claw/Intake Ability	6	8		8		8	
Cycling Efficiency	5	8		9		7	
Ease of Expansion	3	9		7		8	
Durability	3	9		7		9	
Simplicity	2	8		7		7	
Total Score:			244		272		220

Conclusion

The design matrix has identified Option 2 (Cascade) as the best option. Option 2 received a weighted score of 272, with Option 1 and Option 3 receiving lesser scores of 244 and 220, respectively. Now that we have decided on all the elements of our robot, we can move onto designing it in CAD.



08/15/26

Pre-CAD Meeting & Planning

Written By: Jasper

Proofread By: Sam

Proofread On: 08/15/26

Goal: Before starting the CAD process, meet to plan out the design at basic level.

Pre-CAD Meeting

In the Pre-CAD meeting, we will discuss what will and won't be CADed, the general robot plan, key insights to look out for, and what software we will use and why.

CAD Plan

We have determined our robot design to use a continuous lift to reach the tops of the stacks. The continuous lift will use 22W, and our drivetrain uses 55W. That leaves 11W for the claw arm and intake, with 5.5W each. We will CAD the drivetrain, robot skeleton, and plastic pieces. The hardest design will be our claw mechanism, as the geometry will need to be iterated. When we CAD, we will not be CADing most screws, nuts, or spacers. Not only does this space several hours of time, but also reduces the lag in the software.

CAD Approach

We will begin with the drivetrain, then map out the general robot structure. After, we'll move to the toggle mechanism and the intake. From there, we'll CAD the lift, then the claw arm and claw itself. Then comes tuning: ensuring the cup/pin path is optimal, that everything is aligned smoothly, and that all points for assembly are accessible.

Software Choice

We have decided to use Fusion360. Jasper has 4+ years of experience in using Fusion, so it is ideal for him. It is also quite easy to share designs with teammates. Fusion has the ability to render designs and create structural simulations, making it ideal as well for design capability. Furthermore, there is extensive online support for Fusion360, both in VEX and general software usage.



08/16/26

CAD Day 1

Written By: Jasper

Proofread By: Trevor

Proofread On: 08/17/26

Goal: Document CAD progress and highlight key elements.

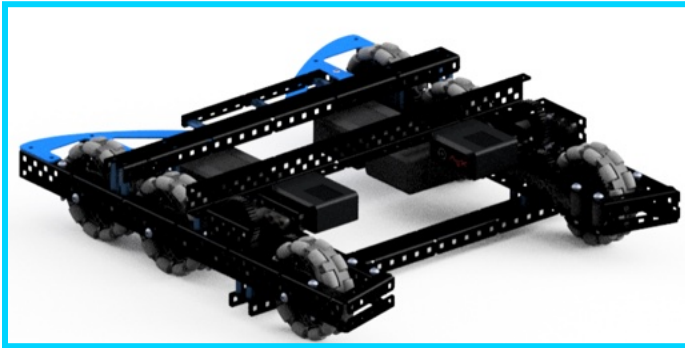


Figure 1.1: Drivetrain Subsystem, Isometric

Gearing

To the right is the drivetrain gearing, in a more visible view. A 5.5W motor is connected to a 3:1 ratio (36T:12T) to convert its 200 RPM to match that of the other 11W 600 RPM. The 3, 48T gears are all attached to a wheel.



Figure 1.2: Drivetrain Subsystem, Gear View

Funneling + Alignment

2, 5-hole 1x1 L-Channels were used to create funnels for the cups and pins to come into the intake. On the other side, two pieces of plastic were used to help align the robot to the goal for scoring purposes. Both the funnels and the alignment will help brace either end of the robot structure.

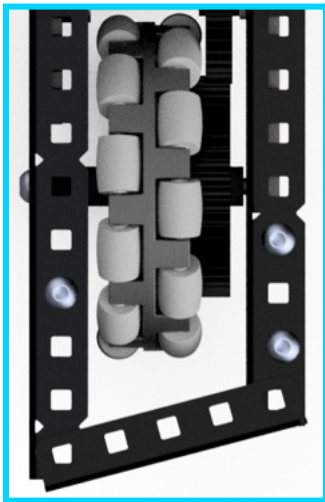


Figure 1.4: Drivetrain Subsystem, Alignment

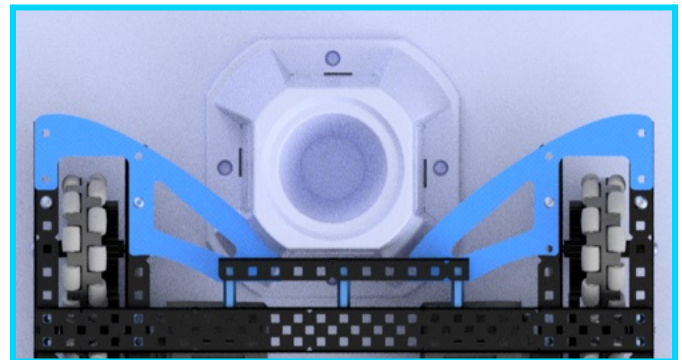


Figure 1.3: Drivetrain Subsystem, Funnels



08/17/26

CAD Day 2

Written By: Jasper

Proofread By: Trevor

Proofread On: 08/17/26

Goal: Document CAD progress and highlight key elements.

Structure and Passive Toggle

A triangle brace was affixed to two diagonal pieces. A plastic brace was added to support a cross-robot standoff, and to add a hardstop for the toggle mechanism. Triangle bracing was chosen for its strength, and so that we can mount the toggles without obstructing the front intake.



Figure 1.1: Toggle + Bracing, Isometric



Figure 1.2: Passive Toggle + Field, Side View

Passive Toggle Explanation

To the left is the passive toggle mechanism. A 1x1 L-Channel contains a standoff. The L-Channel is able to pivot off of the bracing. At velocity, the standoff will contact the toggle rollers on the side of the field. The forward momentum will leverage the roller's slant, causing it to rotate clockwise. The L-Channel is banded downwards to ensure consistent contact.



08/18/26

CAD Day 3

Written By: Jasper

Proofread By: Trevor

Proofread On: 08/19/26

Goal: Document CAD progress and highlight key elements.

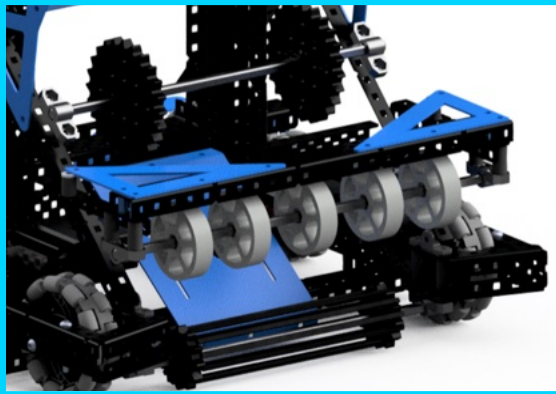


Figure 1.1: Intake Subsystem, Isometric

Intake

A multi-stage intake mechanism was affixed to the triangle structure. Using two triangle-shaped delrin cutouts, the pivotable intake was braced to ensure rigidity. Using flexwheels, the rubber-plastic friction will ensure a clean transfer from field to robot.

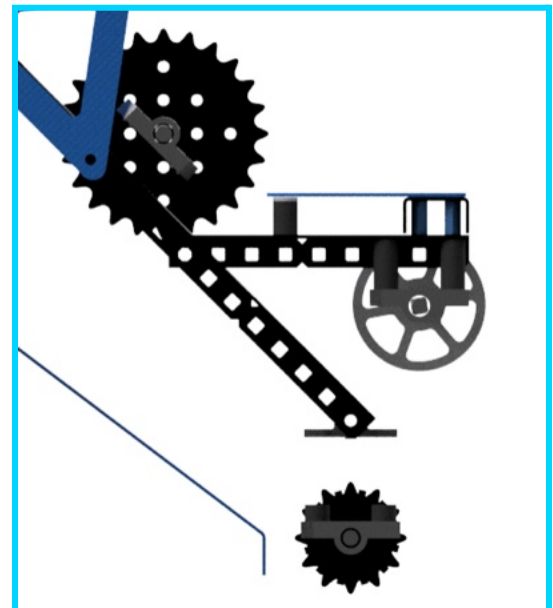


Figure 1.2: Intake Subsystem, Side

Second Stage + Counter Roller

In order to ensure the cup/pin path is smooth and clean, two other rollers were CADed. One larger second stage sprocket-band roller helps push game objects all the way through the robot, to the rear, where it will be picked up by the claw. A smaller sprocket-band roller will ensure clean pick-up from the field.

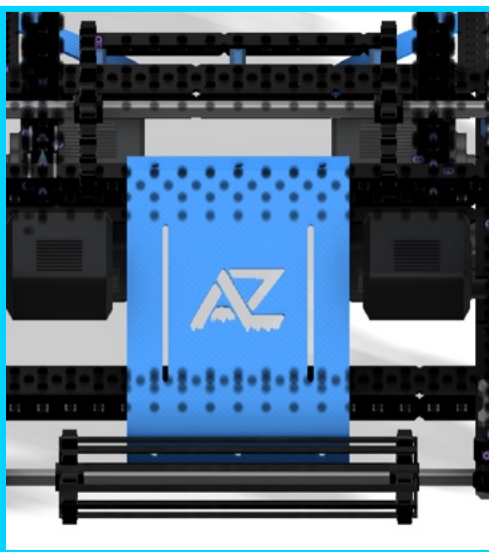


Figure 1.3: Ramp, Front

Ramp

A ramp made of delrin will help smoothly guide the cup/pin to the rear of the robot. It is bent at either end for affixation to metal structure (see Fig. 1.2). Additionally, our logo was cut out from the ramp for a little bit of robot flair.



08/19/26

CAD Day 4

Written By: Jasper

Proofread By: Trevor

Proofread On: 08/19/26

Goal: Document CAD progress and highlight key elements.

Three-Stage Continuous Lift

Note: in this notebook, the words "continuous" and "cascade" are used interchangeably. Though they mean different things, our lift is truly a continuous lift as opposed to a cascade.

The three-stage lift uses 3 sets of 29-hole channels in order to reach a height of ~42 inches. In the brainstorming phase, it was determined we will have a claw arm; this will help push our height to ~48 inches. The lift is mounted from the far rear drivetrain crossbrace, and each stage is progressively closer to the front of the robot. Using a traditional continuous lift, in which a string pulls the bottom of each stage to the top of the previous stage, we can rig string that when coiled extends the lift upwards, and releases the lift when the string is uncoiled.

Winch

In order to coil and uncoil the string, we designed a winch, which essentially has one fixed point of string connection. The winch then rotates with high torque to coil the string around it. The string tension keeps it tight to the winch; the motor locking on the winch ensures it doesn't unfurl. The winch uses two 600 RPM motors. The 22W total was needed to ensure enough power to use 600 RPM. The RPM was designated for the purpose of a fast extension, which is necessary for match play.

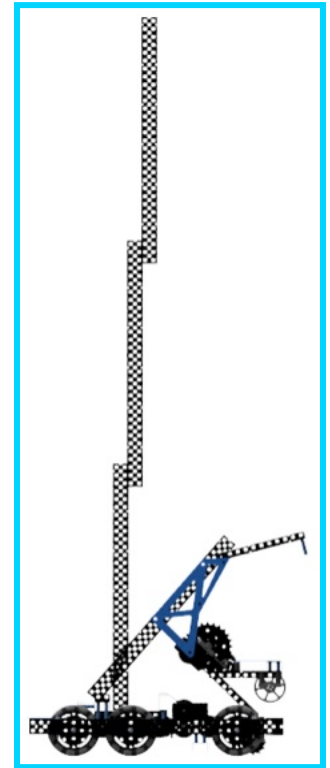


Figure 1.1: Lift, Side

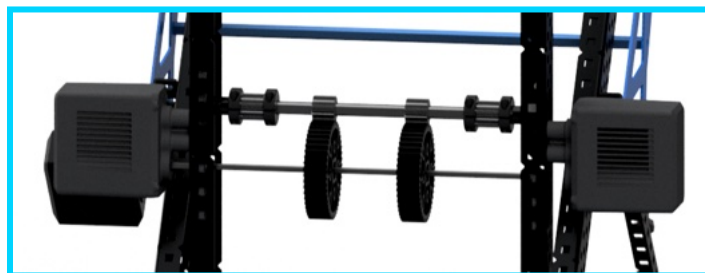


Figure 1.2: Winch, Isometric

stages to rest: without it, the string exerts unnecessary effort keeping the stages in the air. One final note: due to the complex nature of string-rigging, it was determined that it is far easier to determine the correct positions for guides and pulleys using the physical lift, once constructed. This both is time-efficient in CAD (saving many simulations and physics calculations) as well as guaranteeing a more precise mechanism (removing the CAD-real world complications).

Resting Platform and Notes

A "resting platform" was designed using a cut 1x1 L Channel bridging two drivetrain crossbraces perpendicularly, as shown in green, in Fig. 1.3. The resting platform provides a place for the lift

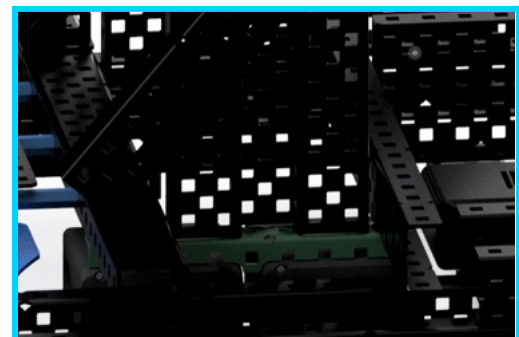


Figure 1.3: Resting Platform



08/20/26

CAD Day 5

Written By: Jasper

Proofread By: Trevor

Proofread On: 08/20/26

Goal: Document CAD progress and highlight key elements.

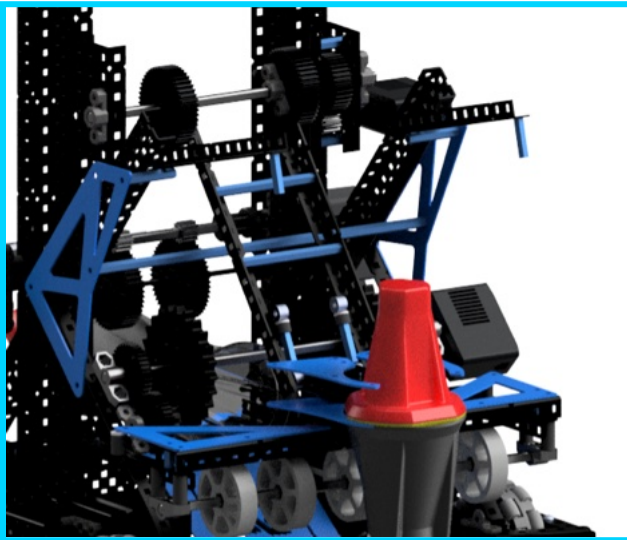


Figure 1.1: Claw Arm, Matchload State

Claw

The claw itself is mounted on a 3-wide channel connected via standoff-collar pairs to the claw arm. The claw consists of two pieces of plastic, folded to imitate four faces, and help circumvent the 12 piece plastic allotment. Using pivot points, a linear guide, and a piston, we can pneumatically power the claw's extension and contraction, as shown. The claw maintains a fixed displacement in relation to the claw arm pivot, meaning that a 180° rotation of the arm will also rotate the claw (and thus cup and pin) by 180°. This allows us to essentially flip the cup and pin, which is necessary based on the current top object of each stack on the field.

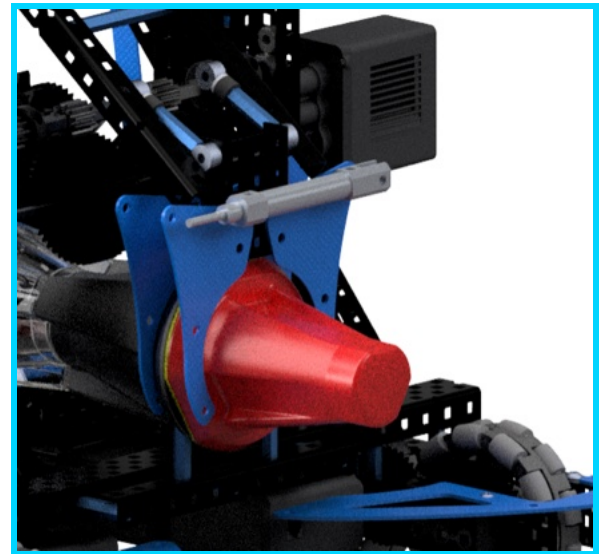


Figure 1.2: Claw, Loading State

Multi-functionality of Claw

The claw was CADED to be able to both follow the standard path (cup/pin are intaked into rear, claw picks up then scores) but also to be able to matchload, via positioning in the front of the robot. Ease of matchload, as discussed earlier, is vital for match play. This multi-functionality simplifies the robot design, as we don't have to create an alternate matchloading mechanism.



08/21/26

Final CAD Remarks & Build Prep

Written By: Sam & Jasper

Proofread By: Trevor

Proofread On: 08/22/26

Goal: Outline key aspects of the robot design and prepare to start building.

CAD Challenges

Just as a freshman might upon entering high school, we found geometry especially challenging. In our case, robot geometry. To ensure the robot's structure, specifically the triangular bracing aligned with each grid hole, we had to try many combinations in CAD. Though it is possible to use trigonometry and other algebraic methods to determine these placements, they often take longer, are more error-prone, require research into material thickness and dimensions, and are, in general, overcomplicated. Trial-and-error, though less elegant, was the most time-efficient solution to our geometric constructions. Determining the geometry of several items on the bot, such as the toggles (which also act in relation to bot position on the field and field elements), took a lot of time.

By far, the most difficult problem was the claw arm and claw geometry, the combination of which took several hours (and an attempt at algebraic methods, which added unnecessary complexity). Further CAD challenges included the correct geometry of folding plastic (quite hard to CAD a folded piece without a beginner's introduction to topology, which of course Jasper has taken). In addition, the Fusion software itself presented issues: a timeline issue, a jointing issue, and save/cross-sharing issues, all of which took time to resolve. Though we had setbacks, the overall design was a massive success, and we are excited to see the build result.

CAD Successes

Our problems were many, but so were our successes. We have created a design that we believe will be highly competitive, while maintaining simplicity. One thing we have learned from our years of experience at VEX: simpler bots are more consistent; consistency is key to success. Our "the simpler, the more successful" model helps guide our build process. That is not to say this bot is completely simple; rather, it meets all our standards in what we believe is a very simple approach. Such as matchload time, for instance — instead of an overcomplicated roller mechanism, we simply utilize our already existing claw to grab matchloads.

Overall, the major successes came in the claw geometry (ideal for quick matchloads and scoring), the bracing geometry (major structure triangle, three drivetrain crossbraces), and center of gravity (heavy near bottom, very light at top, ideal for not tipping over).

In the future, we'd like to be a bit quicker with the design process. From section A, CAD Challenges, we've learned from our mistakes and adversities, and will hopefully be quicker with those types of problems should they ever arise.

Photos

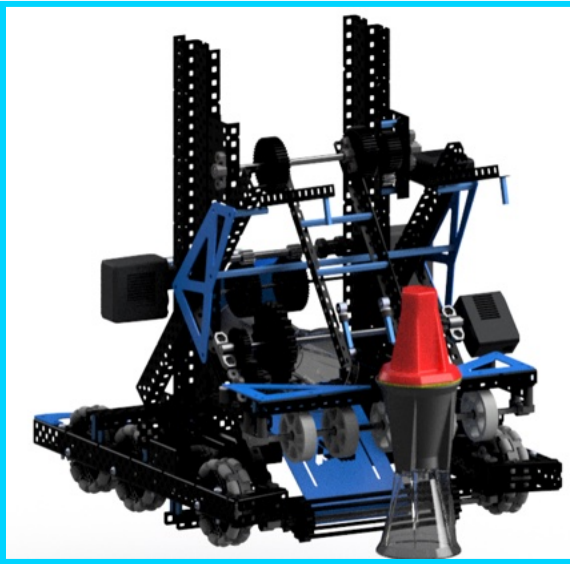


Figure 1.1: Isometric View

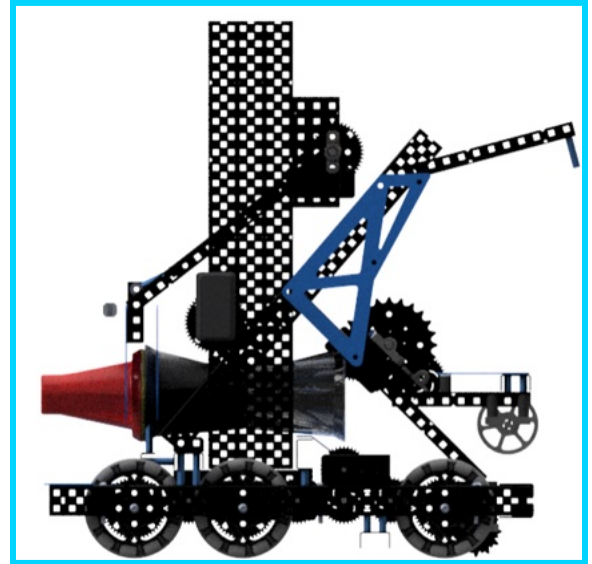


Figure 1.2: Side View

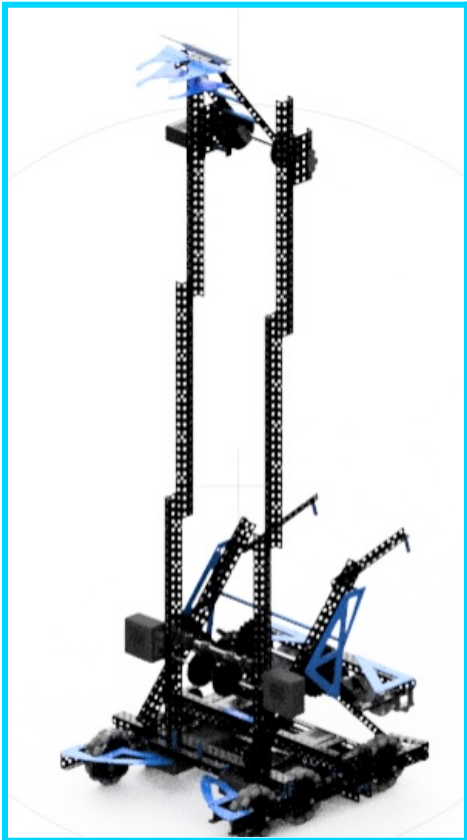


Figure 1.4: Extended Lift, Back View

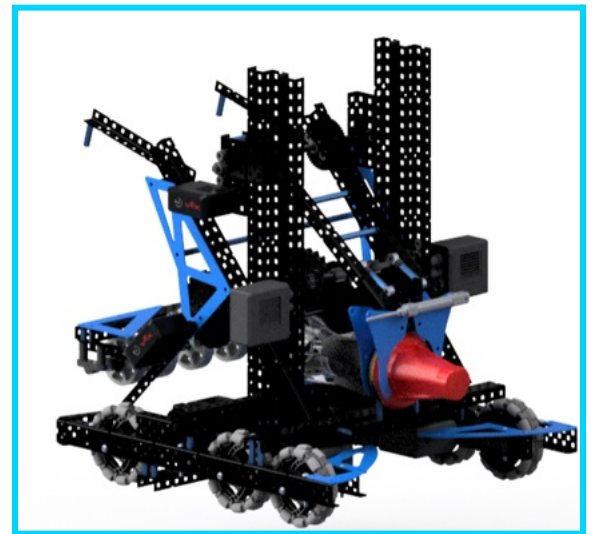


Figure 1.3: Back View



08/24/26

Build: Day 1

Written By: Trevor

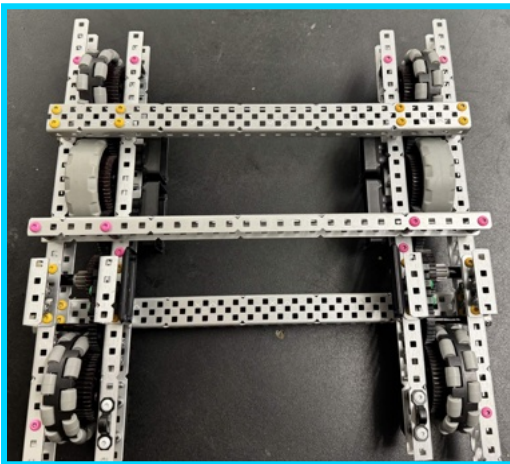
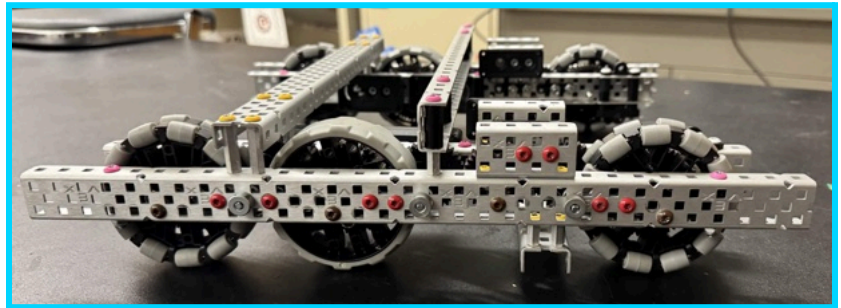
Proofread By: Jasper

Proofread On: 08/25/26

Goal: Begin building the drivetrain.

Drivetrain Construction

Today, we began constructing our drivetrain. We used screw joints for each wheel to reduce friction and allow smooth rotation, with shaft collars securing the axles in place. For the drivetrain frame, we connected two 35-hole c-channels to two 33-hole c-channels, which creates a rigid base for mounting our gears, wheels, and other drivetrain components.

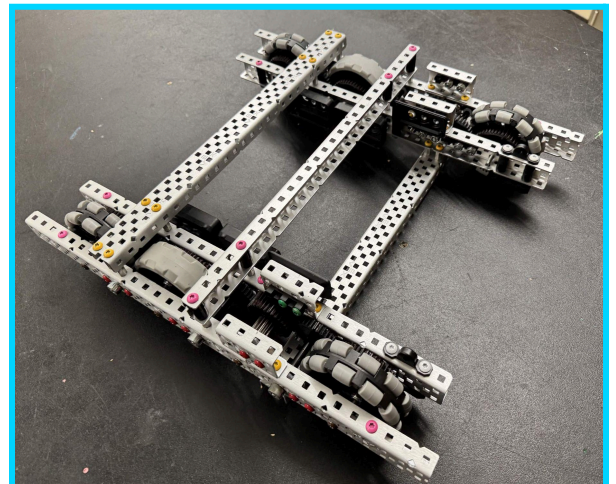


Cross Brace

We used three 27-hole c-channels as cross braces to increase the drivetrain's structural stability. Two were mounted horizontally across the frame, one on the top and one on the bottom. The third was mounted on the top upright across the frame, providing additional stability and resistance.

Day's Progress

By the end of the day, we finished the entire drivetrain's main base. However, after building, we realized some of the spacing is a bit off. When we screw it in it gets compressed, and after testing briefly we realized it has really bad friction. Tomorrow, our next step is to fix spacing and improve friction.





08/25/26

Build: Day 2

Written By: Helena

Proofread By: Jasper

Proofread On: 08/26/26

Goal: Finish building the drivetrain.

Today, we continued building the drivetrain and ultimately finished it. We took no new photos because the robot's core structure looked mostly identical. Instead, we focused on fixing both major and minor friction and spacing issues.

This was especially important because the drivetrain is one of the most essential parts of our robot and one of the hardest to fix later in the build process. We wanted to make sure every part moved as smoothly as possible with minimal friction. We spent the day carefully checking for even small amounts of friction, finding the source, and making adjustments. Our next step will be to fully test the drivetrain and make sure it runs smoothly and consistently.



08/26/26

Test: Drivetrain Friction

Written By: Trevor

Proofread By: Helena

Proofread On: 08/27/26

Goal: Test the friction of each side of the drivetrain to make sure the drivetrain has minimal friction and will drive straight.

Procedure

1. Suspend the drivetrain in the air.
2. Spin a wheel on one side of the drivetrain.
3. Start a stopwatch the moment the wheel begins spinning.
4. When the wheels on that side of the drivetrain stop spinning, stop the stopwatch and record the time.
5. Repeat four more times on the same side of the drivetrain.
6. Repeat steps 1-5, but with the other side of the drivetrain.
7. Calculate the average spinning time for both sides of the drivetrain.

Criteria

- The average spinning time of both sides of the drivetrain must be >6.5 seconds.
- The average spinning time of both sides of the drivetrain must be within 0.5 seconds of each other.

Results

	Trial 1	Trial 2	Trial 3	Trial 4	Trial 5	Average
Left Side	6.85	7.14	6.98	6.91	7.19	7.05
Right Side	6.97	6.92	7.13	6.84	7.04	6.98

Conclusion

The average spinning time for the left side of the drivetrain was 7.05 seconds, while the right side averaged 6.98 seconds. This small difference of just 0.07 seconds is well within our 0.5 second tolerance, indicating that both sides have minimal friction and the difference is unlikely to affect how straight the drivetrain drives. As both sides also exceeded the 6.5 second average, our drivetrain meets the established criteria and does not need any improvements/changes.



08/27/26

Build: Day 3

Written By: Helena

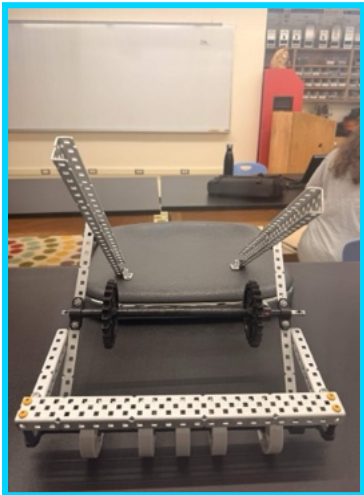
Proofread By: Jasper

Proofread On: 08/28/26

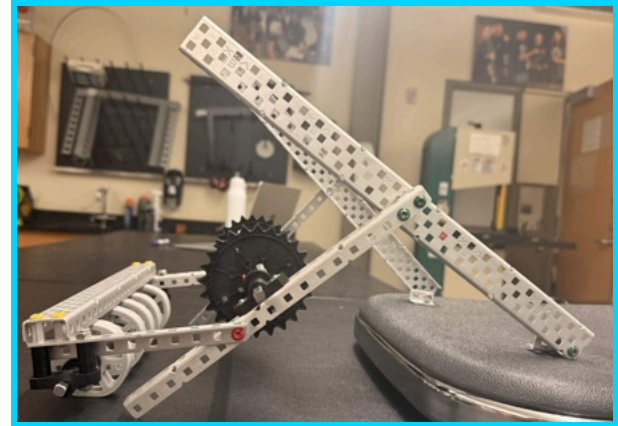
Goal: Begin building the intake.

Intake Mechanism

Today, we began building the front intake mechanism. We added a



row of five flex wheel rollers that will help pull and guide game objects into the robot. Behind the rollers, we added two 24T sprockets connected via a high strength axle, mounted on robot structure. We also used two matching support c-channels on each side to keep the intake sturdy and reduce flex. The intention is to extend rubber bands across the sprockets to help pull the game objects further in the robot.



Structure

We added several c-channel supports around our intake to strengthen the structure and keep both sides even. The two long c-channels will attach to the drivetrain base at the angle shown, helping support the intake as it moves, and laying the foundation for the toggle mechanism. The larger c-channels are also connected by a high-strength axle, with proper spacing and two 24T sprockets to keep both sides aligned and moving together.

Day's Progress

Today, we finished building the entire intake mechanism. After informally testing it, we found no friction or movement issues, and the rollers and supporting structure held well. Our next step is to attach the intake to our current drivetrain base tomorrow and begin testing how the two systems work together.



08/31/26

Build: Day 4

Written By: Helena

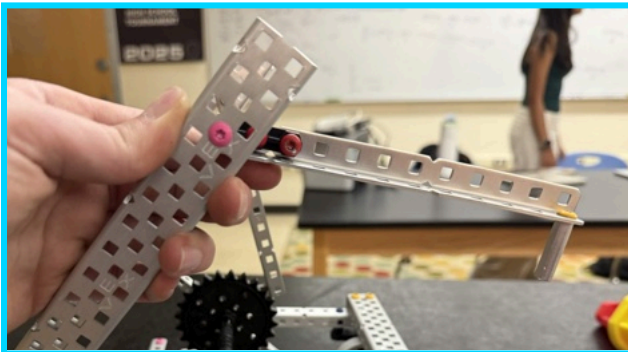
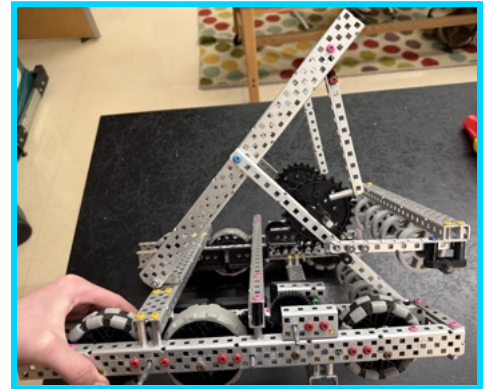
Proofread By: Jasper

Proofread On: 09/01/26

Goal: Begin attaching the intake.

Intake Structure

Today, we attached the intake structure from Build Day 3 to our drivetrain. The intake pivots on a long, diagonally-mounted c-channel in the back of the robot, and is braced by a smaller L-channel triangularly. The flexwheel rollers extend from two L-channels on screw joints, and are braced perpendicularly by a c-channel.



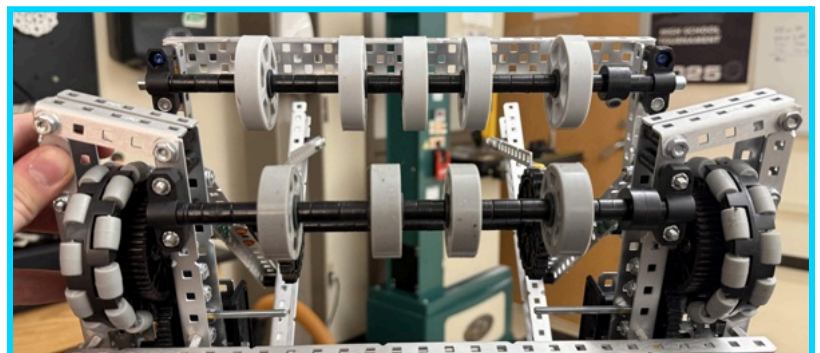
Intake Mechanism

For the intake mechanism, we replaced the sprocket shown in our CAD on the bottom counter roller with flex wheels because the sprocket was too large and contacted the ground. The flexwheels give us more ground clearance, but they are still slightly too large, so we will cut them down after more testing rather than wasting parts immediately. We also built the toggle mechanism

using 2 small L-channel arms with 2 standoffs at the end designed to push the field toggle upward and rotate it to the correct color. Initial testing showed us that the toggle mechanism is not working fully yet, so the design will need adjustment.

Day's Progress

Our next steps are to continue testing the intake, determine the correct gearing, adjust the bottom counter roller if needed, and redesign the toggle so it contacts and flips the field piece consistently. After those mechanisms are working reliably, we will begin adding the cascade.





09/02/26

Build: Day 5

Written By: Helena

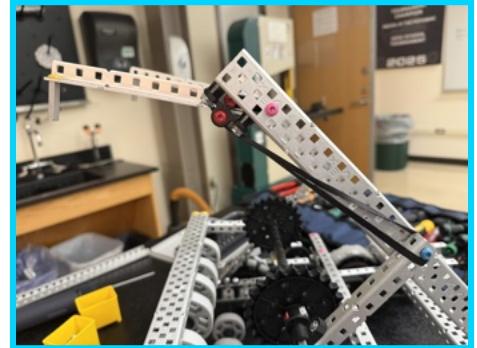
Proofread By: Jasper

Proofread On: 09/02/26

Goal: Fix/tune toggle mechanism

Toggle Mechanism

Our toggle mechanism is the mechanism we will use to flip toggles to our alliance's color. This is important because toggles control the scoring of yellow pins and can create large point swings during a match.



Toggle Mechanism

Our current design uses two L-Channel angle pieces attached to our main structural c-channels. Each has a downward-facing standoff, angled to be parallel to the bottom of the toggle, that will contact the toggle and push it upwards as we drive into it. The rubber bands are temporary for testing and will later be replaced with stronger latex tubing. We will continue tuning the mechanism's height, angle, and hardstops until they can consistently flip the toggle.



Day's Progress

Today, we completed the primary design of our toggle mechanism and will keep the current setup for now. Later, we will replace the rubber bands with stronger and more durable latex tubing. Our next steps are to continue testing the mechanism for consistency and begin building the lift.



09/03/26

Test: Toggliers

Written By: Trevor

Proofread By: Helena

Proofread On: 09/02/26

Goal: Test our “toggliers” to ensure they can reliably and quickly change the orientation of the toggles.

Procedure

1. Ensure the toggle is properly seated in its mounting.
2. Push the left toggler into the toggle.
3. Document “yes” or “no” depending on whether the toggle flipped and was properly seated.
4. Repeat steps 1-3 6 more times, and calculate the rate the toggle flipped and was properly seated.
5. Repeat steps 1-4, but with the right toggler and both toggliers.

Criteria

- The toggle rate for both sides must be $\geq 90\%$.
- The toggle rate for the left/right side must be $\geq 80\%$.

Results

	Trial 1	Trial 2	Trial 3	Trial 4	Trial 5	Trial 6	Trial 7	Trial 8	Trial 9	Toggle Rate
Left Side	Yes	Yes	Yes	Yes	Yes	Yes	Yes	No	Yes	89%
Right Side	Yes	Yes	No	Yes	Yes	Yes	Yes	Yes	Yes	89%
Both Sides	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	100%

Conclusion

The toggle rates for the left side and right side were both 89%, and the toggle rate for both sides was 100%. These rates both meet our requirements and demonstrate superb reliability. A majority of the time Sam (our driver) will be able to hit the toggle head on with both toggliers, meaning the toggle will hopefully always flip. However, we know sometimes ideal conditions don't exist, which is why we tested with only the left and right toggliers too. Having this high reliability will be extremely beneficial in matches as Sam won't have to worry about hitting the toggle numerous times or wasting too much time with it.



09/05/26

Build: Day 6

Written By: Helena

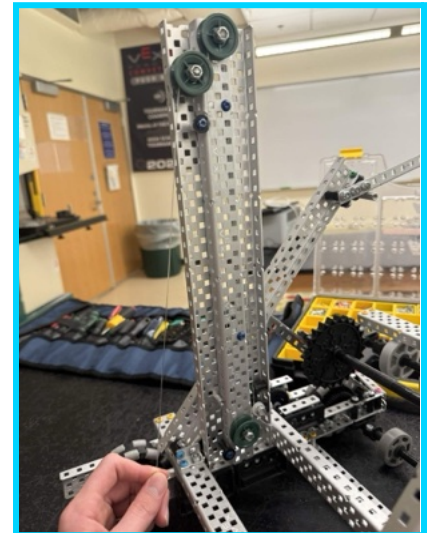
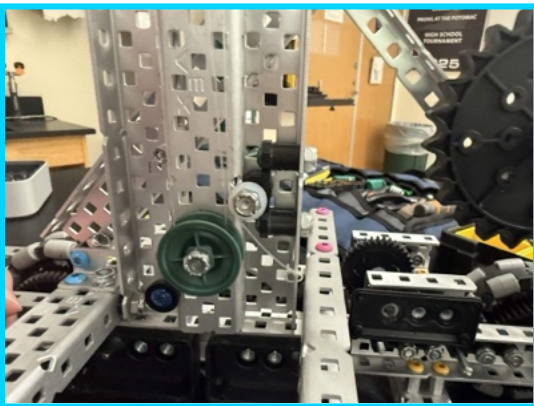
Proofread By: Trevor

Proofread On: 09/03/26

Goal: Begin building the cascade.

Cascade Structure

Today, we began building the cascade. We started with only the left side to make sure the design was accurate, reliable, and easily replicable before building the other side. Much of our time was spent determining the proper spacing between components, since even small spacing errors could cause friction or misalignment once the cascade begins moving.



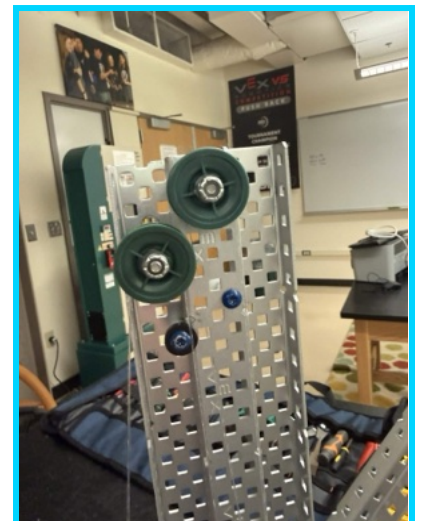
Cascade Mechanism

In our notebook, we will often refer to our lift as “cascade” or continuous. Both of these will be used interchangeably, although they have slight differences. In our case, we have actually built a continuous lift. Rather than having separate pieces of string that loop over each other and pull the stages separately, we have only one piece of string per side. At the bottom of our top stage, we tie

the string, and then loop it over the top pulley on the second stage, around the bottom pulley on the second stage, up over the pulley on the bottom stage, and then tie it onto the winch. This single piece of string is a lot better, because it allows the cascade to be pulled up all at once, making it smoother and faster. It also makes the robot a lot cleaner, so we don’t have to worry about multiple pieces of string tangling with each other or tangling with chords and pneumatic tubing.

Day’s Progress

Our goal for tomorrow is to continue building the cascade and hopefully finish it. When building the left side, we will make sure it accurately matches the completed right side, including spacing, c-channel lengths, and screw placement, so both sides remain aligned and the cascade can operate as smoothly as possible.





09/06/26

Build: Day 7

Written By: Helena

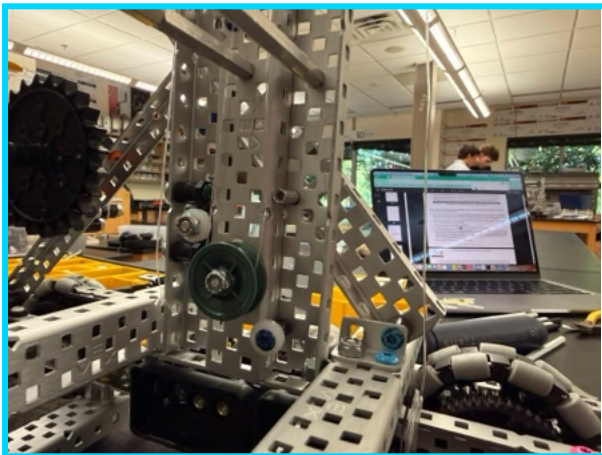
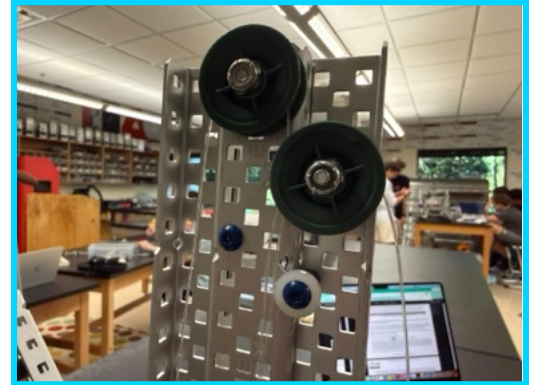
Proofread By: Jasper

Proofread On: 09/04/26

Goal: Continue building the cascade.

Cascade Structure

Today we continued to build the cascade lift. We replicated the previously built left side and made minor adjustments to pulley wheel placement and line guiding while building. We also added a small amount of nonaerosolized lithium grease to the metal on metal location to reduce friction.

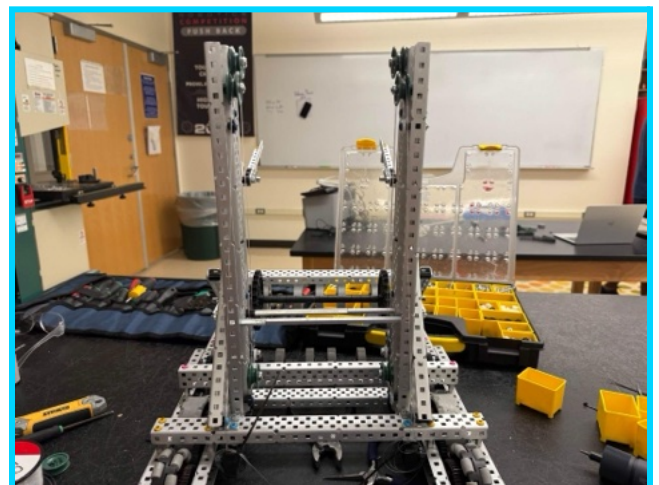


String Attachment

For our cascade string, we used fishing line because it is extremely thin and lightweight, helping us minimize unnecessary weight on the robot. We will continue observing its performance during testing and adjust or replace it if needed.

Day's Progress

We fully completed the cascade by the end of the day. One challenge was adding and properly routing the string on the side we had previously built, then figuring out how to replicate the same setup on the second side. Once we determined the correct string placement and spacing, building the second side went much faster because we could directly replicate the first. Overall, the rest of the build went smoothly.





09/07/26

Test: Cascade Friction

Written By: Trevor

Proofread By: Helena

Proofread On: 9/10/26

Goal: Test our cascade to ensure there is no friction or “sticky” spots.

Procedure

1. Slowly pull both strings simultaneously until the cascade is fully extended.
2. Observe any spots where the cascade sticks or jumps.
3. Slowly let the string and cascade down until the cascade is fully seated against the bottom hard stop.
4. Observe any spots where the cascade sticks or jumps.
5. Document the success with either a “yes” or “no”. “Yes” entails no sticking, “no” entails friction/sticking.
6. Repeat steps 1-5 while pulling at a moderate and fast speed.

Criteria

- The overall success rate must be $\geq 90\%$.
- The success rate must be within 15% for all speeds.

Results

	Trial 1	Trial 2	Trial 3	Trial 4	Trial 5	Trial 6	Trial 7	Trial 8	Trial 9	Toggle Rate
Slow	Yes	Yes	Yes	Yes	No	Yes	Yes	Yes	Yes	89%
Moderate	Yes	Yes	Yes	Yes	Yes	Yes	Yes	No	Yes	89%
Fast	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	100%
Overall										93%

Conclusion

Overall, we are quite happy with the current performance of our cascade. We met both of our criteria, with the overall success rate being 93%, and the three success rates being only 11% apart. We know this test is a bit vague as of right now given the lack of quantitative data, but this is just a preliminary test to ensure we have no major weaknesses. We will perform more extensive testing once the entire system is complete. Our next steps will be to add the clamp (which will add significant weight to the cascade) and add the motorized winch for the cascade (which will allow us to test actual speed and consistency).



09/10/26

Build: Day 8

Written By: Helena

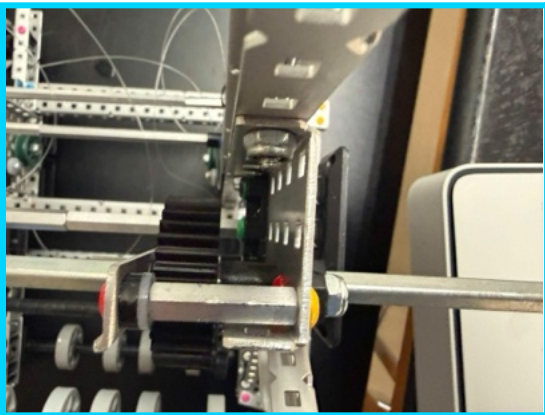
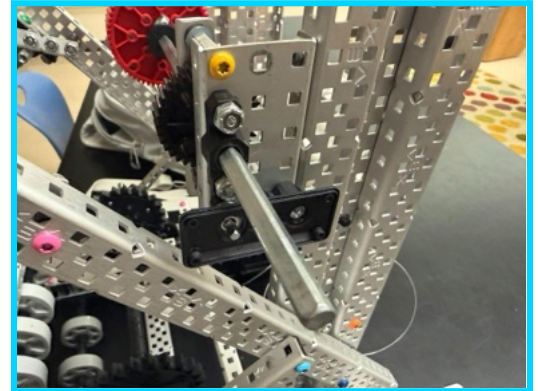
Proofread By: Jasper

Proofread On: 09/09/26

Goal: Begin building the claw arm.

Claw Arm Structure/Bracing

We mounted our claw arm structure to the final stage of the cascade. We then added a standoff across the two sides of the arm structure to provide additional bracing and reduce flex. We then drilled out a bearing for a high strength axle.

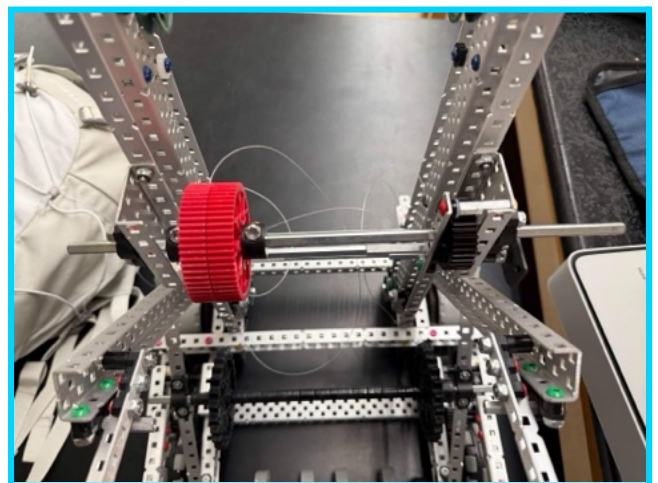


Gearing/Motor Placement

We cut a 3-wide c-channel and shaved down one edge almost completely, to create a flat-bar shape that had more strength and resistance to bending than a regular VEX flat bar. Our gear ratio is 1:3, and we used one 5.5W motor to power the mechanism. We chose this gear ratio to prioritize torque and give our arm controlled movement.

Day's Progress

Today, we added the bracing and mounts for the claw. This took some time because we had to drill larger holes into the metal, determine the correct bearing placement, and make sure the entire structure was as compact as possible. Our next steps are to finish the claw arm itself and hopefully attach both the structure and arm to the robot.





09/11/26

Build: Day 9

Written By: Helena

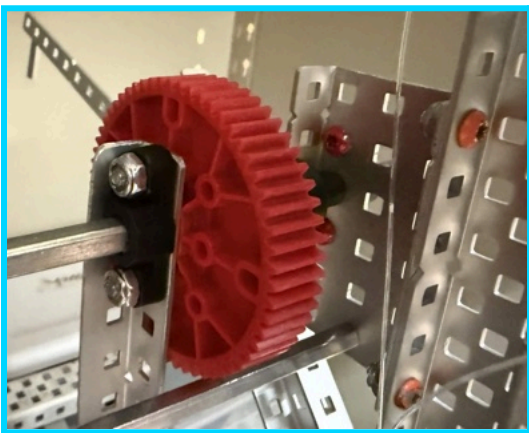
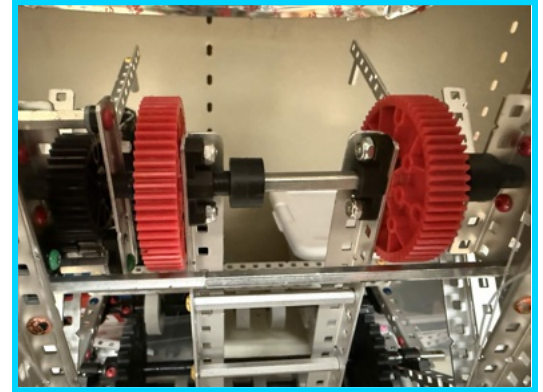
Proofread By: Trevor

Proofread On: 09/11/26

Goal: Finish building the claw arm.

Claw Progress

We attached cut 3-wide c-channels to the high-strength bearings and gears. We chose cut 3-wide's because the extra metal on one side makes it stronger and its proportions make it easier to mount the high-strength bearings. The gears connect to the high-strength axle, allowing the claw arm to swing when powered by the motor.



Claw Testing

Adding the claw arm, especially the high-strength axle and motor, added a significant amount of weight to the final stage of the cascade. Because of this, we did some brief, informal testing to make sure the added weight did not create any new issues. So far, the cascade still moves smoothly, and the fishing line is able to handle the additional weight without any problems.

Day's Progress

Today we finished building the claw arm. We will add the winch to lift the cascade before adding the actual claw, so we can make sure the cascade functions properly and determine whether any larger structural changes are needed before continuing.



09/12/26

Build: Day 10

Written By: Helena

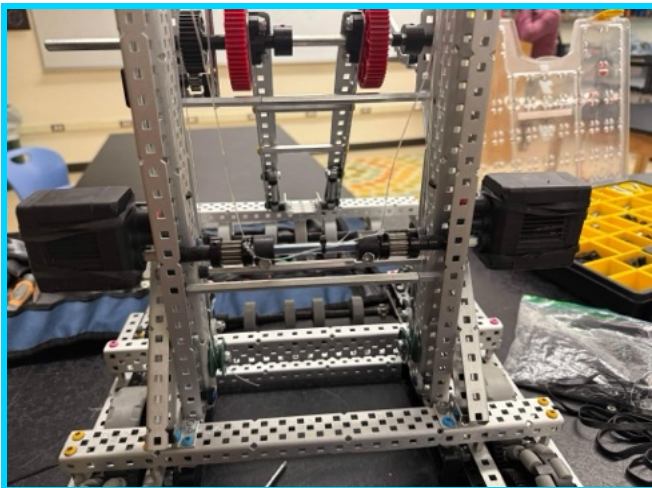
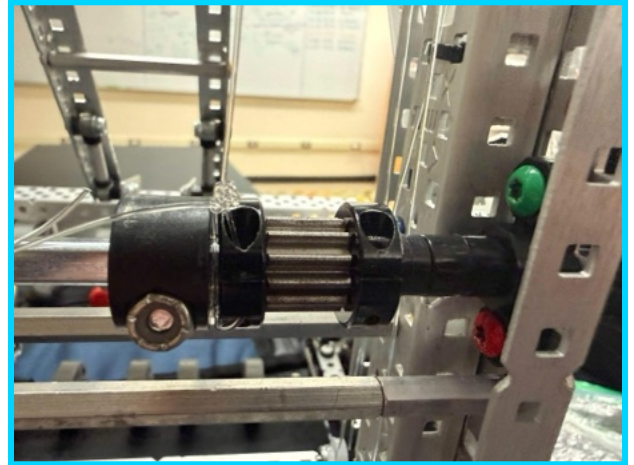
Proofread By: Trevor

Proofread On: 09/11/26

Goal: Build the winch & add the pin holder.

Build

Today, we built the winch. We used a 12T gear with low-profile high-strength shaft collars on each side. To keep the gear and collars from shifting along the high-strength axle, we added a larger high-strength shaft collar toward the inside. The axle is mounted to the c-channel using a high-strength bearing block for support. We chose fishing line to wrap around the collars because it is lightweight but strong enough to support the weight our bot can lift.



How It Works

The winch is powered by 22W (11W per side, on a 600 RPM motor) using a high strength axle spanning across the robot, on the bottom stage of the continuous lift. Utilizing two low-profile high-strength shaft collars, we affixed the fishing line to the outermost collar. This way, when the winch is spun, the fishing line coils around the 12T gear, pulling the lift upwards. A reversed rotation allows it to move down.

Day's Progress

After completing today, we will test our winch to ensure that it can effectively pull our cascade up and down. Our next steps are to attach the claw we built on day 9.



09/15/26

Build: Day 11

Written By: Jasper

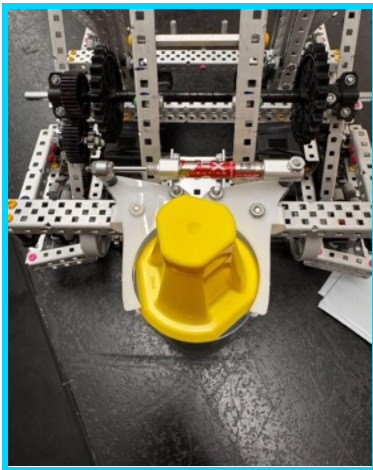
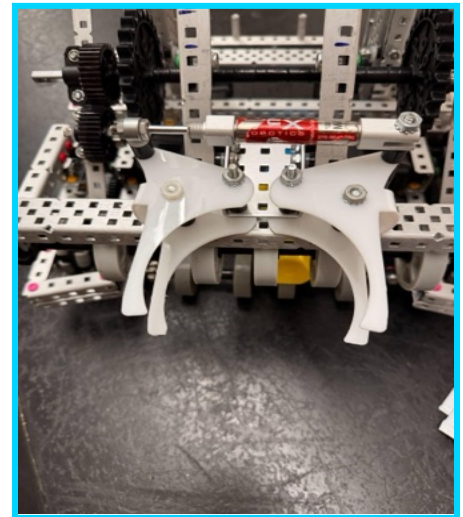
Proofread By:

Proofread On:

Goal: Attach the claw to the claw arm.

Claw Plastic

The claw plastic utilizes a screw joint on the claw arm to pivot in and outwards. A piston pushes and pulls two alternate points to clamp the cup and pin, as shown on the right. The claw plastic is folded over itself in order to minimize the amount of plastic used. The claw is circular to fit the cup/pin combination.



In Action

Shown here is the claw in action; the piston extends to force the claw itself to contract, grabbing the cup and pin. The claw can then rotate and move, keeping the cup and pin secure the entire time. The circular design of the claw is good for the pin, and we could manually hold it in place. However, later testing with the cup + pin yielded other results.

Day's Progress

After testing, we have determined that this claw design is not suitable for the most optimal build. The claw plastic can not grab the pin with enough grip, and does not wrap around the pin as much as it should. The claw will have to be reworked the next day to alter its grip. We believe the best next step will be to lengthen the claw arms, and add a small linear slide that helps keep the claw segments fully in place.



09/16/26

Build: Day 12

Written By: Helena

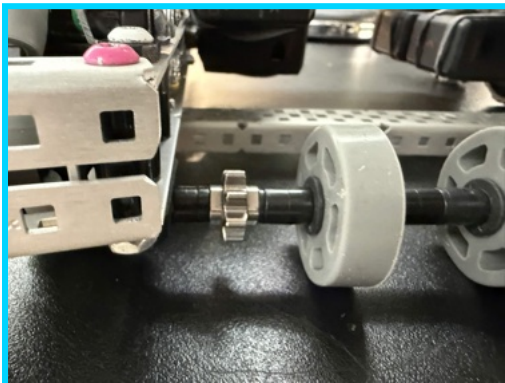
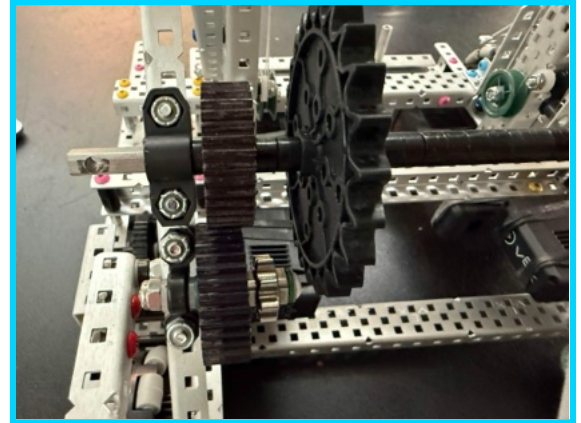
Proofread By: Jasper

Proofread On: 09/16/26

Goal: Connect all intake stages and counterrollers.

Chaining the Intake

Today, our goal was to chain together the intake. Before adding the chain, we first wanted to make sure there was as little friction as possible throughout the intake. We manually rotated and moved parts of the intake, checking the axle, gears, and rollers for any resistance or misalignment. After testing, we found no noticeable friction, meaning we could move forward without making any major adjustments. We then decided to use 6P chain to connect and power the different parts of the intake.



Chain Type

We decided to use a lower profile 6P chain because it gives us more space to fit other mechanisms within our robot. Its smaller size creates more points of contact, and thus more surface area/friction which helps reduce the chance of slipping. Overall, the 6P chain is thinner, lighter, and more compact, making it a better fit for our intake design (and most chains in the future as well).

Day's Progress

Overall, after fully finishing the intake, we decided that there are improvements to be made. Currently. Although it works, we are struggling with trying to move the pin up our bot consistently and with enough power to reach the back of the robot. Thus, we will focus on ramp adjustments, looking into the possibility of intake funnels, and tuning intake and stage placement. With the chaining blueprint set up, we are ready to start fully testing the intake → claw pipeline that we will rely on in match play.



09/18/26

Code: Drivetrain & Intake Control

Written By: Trevor

Proofread By: Jasper

Proofread On: 09/14/26

Goal: Program the basic functions of the drivetrain and intake.

Defining Drivetrain & Intake Motor Ports

```
ez::Drive chassis(
    {-14, -15, -16},    // Left Chassis Ports
    {11, 12, 13},    // Right Chassis Ports
    7, 3.25, 450    // IMU Port, Wheel Diameter (in), Wheel RPM
);
```

Above is the function, found in the “main.cpp” file, that communicates to the code what the characteristics of our drivetrain are. In the second line, we put the ports that the motors for the left side of the drivetrain are connected to: 14, 15, and 16. The third line is the same thing, but for the ports for the right side: 11, 12, and 13. The fourth line is different to the last two, as it is where we define the port that is connected to the inertial sensor, the diameter of our wheels, and the RPM of our drivetrain. We have not yet connected an inertial sensor to our robot, so we just put in a placeholder number for now.

```
inline pros::Motor intake (-21);
```

This line, found in the “intake.hpp” file, defines the port for the intake motor, which is port 21.

*** Note: A negative port number indicates that the direction of the respective motor is reversed.**

Basic Functions for Driver Control

```
void IntakeControl() {
    if (master.get_digital(DIGITAL_R1)) {
        intakeMove(127);
    }
    else if (master.get_digital(DIGITAL_R2)) {
        intakeMove(-127);
    }
    else {
        intakeMove(0);
    }
}
```

This is the intake control function for driver control, located in “intake.cpp.” The first if statement in this function makes it so that if R1 is pressed on the controller, the intake spins forwards to collect pins and cups.

The second if statement makes it so that if R2 is pressed, the intake spins backwards. Lastly, the final else statement just stops the intake from spinning when R1 or R2 isn't being pressed on the controller.

*** There are comments in the code that explain what each statement means, but I removed them when I pasted them here because I didn't want the code to take up too much space in the entry.**

```
void opcontrol() {
    chassis.drive_brake_set(MOTOR_BRAKE_COAST); // Switch motor brakes to coast

    while (true) {
        ez_template_extras(); // Built in EZ-Template extras
        chassis.opcontrol_tank(); // Activate tank control
        IntakeControl();

        pros::delay(ez::util::DELAY_TIME); // Slight delay to prevent cpu overload
    }
}
```

This is the main driver control function, located in "main.cpp." The main purpose of this function is to recall separate subsystem control functions, such as the intake control function I just made, in an infinite loop during the driver control period. The infinite loop is that "while (true)" statement, and the individual functions need to be running infinitely so the code doesn't just check the if statements one time, but for the whole duration of the driver period. The first main function in this loop is the "chassis.opcontrol_tank" function, which sets the drive mode to tank, which is where the joysticks on the controller control their respective sides of the drivetrain (left with left, etc.). There are other drive modes, such as arcade, but Sam's preferred mode is tank. As I mentioned before, the intake control function is also in this loop.

Currently, the only important thing outside of the infinite loop is the "chassis.drive_brake_set" function, which is where we set brake mode for the drivetrain motors. It is currently set on coast, which is Sam's preferred mode as well.



09/19/26

Code: Cascade Control

Written By: Trevor

Proofread By: Jasper

Proofread On: 09/14/26

Goal: Program the basic functions of the cascade.

Defining Cascade Motor Ports

```
inline pros::Motor cascadeLeft (-2);
inline pros::Motor cascadeRight (3);
```

Basic Functions for Cascade Control

```
void CascadeControl() {
    if (master.get_digital(DIGITAL_L1)) {
        cascadeLeft(127);
        cascadeRight(127);
    }
    else if (master.get_digital(DIGITAL_L2)) {
        cascadeLeft(-127);
        cascadeRight(-127);
    }
    else {
        cascadeLeft(0);
        cascadeRight(0);
    }
}
```

This is the cascade control function for driver control, located in “cascade.cpp.” The first if statement in this function makes it so that if L1 is pressed on the controller, the cascade motors spin forwards to raise the cascade. The second if statement makes it so that if L2 is pressed, the cascade motors spin backwards, lowering the cascade back to its hard stop. Lastly, the final else statement just stops the motors from spinning when L1 or L2 isn’t being pressed on the controller.

*** There are comments in the code that explain what each statement means, but I removed them when I pasted them here because I didn’t want the code to take up too much space in the entry.**

Now that we have the main code functions, we will need to add some more advanced functions. We’ll need a bit more testing before we decide our final approach, but we think we will add either limit switches or rotation sensors for the cascade. These would allow Sam to simply hold L1 to raise the intake, and not have to worry about pulling the cascade too high and potentially breaking it. We have hardstops, but adding an extra code barrier will help to make sure nothing breaks.



09/20/26

Build: Day 13

Written By: Helena

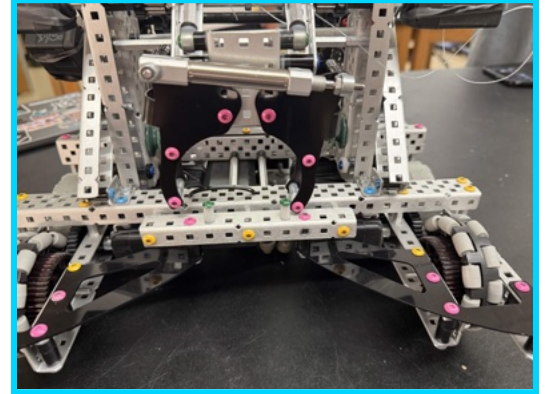
Proofread By: Jasper

Proofread On: 09/17/26

Goal: Redesign and optimize the claw plastic

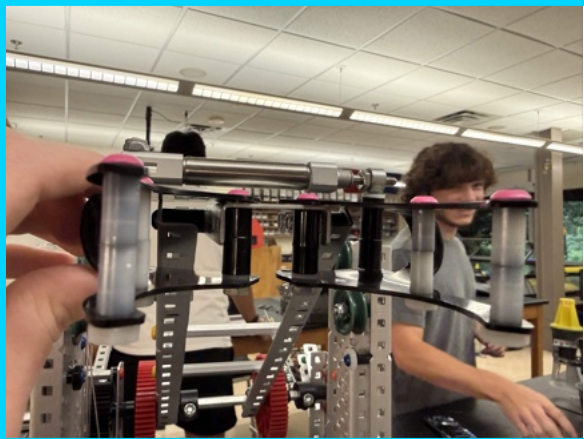
Claw V2

Our initial claw plastic did not quite have the grip or range of motion we desired. As such, we decided to redo the claw plastic. To build our claw, we started by CADing the plastic pieces. Once the plastic was cut, we folded each piece using a heat gun. We aligned the plastic with the edge of a table, secured it with a clamp, and heated it while bending it upward until it reached the shape shown in the photo. We used two half-inch spacers to maintain consistent spacing between the two plastic pieces. As shown, the two folded pieces curve inward toward the center of the claw, helping guide game objects into the mechanism. Each piece of plastic has a linear cutout for a screw to follow (explained in depth on our [CAD Day 5 page](#)) that helps guide the motion of each side of the claw inwards and outwards.



Grip

We used two half-inch spacers between the plastic pieces to create the correct width for gripping a pin. The claw is controlled by a small pneumatic piston, which pulls the two sides inward far enough to securely grasp and hold the pin without allowing it to slip out. As shown in the photo, the piston is mounted between the two plastic claw pieces using plastic pieces that position it directly in line with the claw. This keeps the movement centered and allows both sides of the claw to close evenly around the game object.



Day's Progress

Today, we successfully finished building the claw. In the coming days, we will focus on testing it, making sure its performance is consistent and that it can reliably grasp and hold game objects. We will also begin making final adjustments to the rest of the robot and checking for any remaining issues. Important next steps include completing the wiring, adding the remaining motors, mounting the brain, and making sure all electronics are securely placed and accessible.



09/21/26

Build: Day 14

Written By: Sam

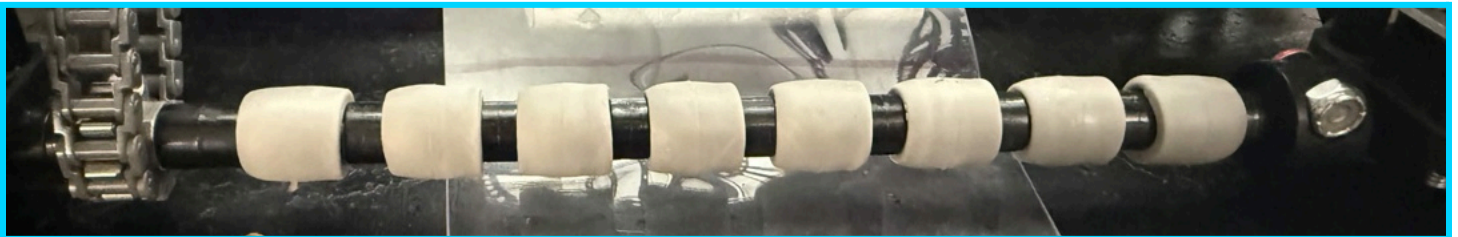
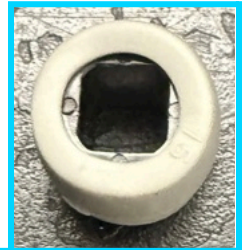
Proofread By: Trevor

Proofread On: 09/22/26

Goal: Work on the intake system and start wiring the robot.

Intake:

We experimented with a few designs for counter-rollers. These are rollers on the bottom side of the intake that help lift to the cups that are hard to pick up. While running the intake, we noticed our counter-rollers were too thick for the cup to lift over. So we decided to try using a thinner material than cut flex wheels to roll the cups into the intake. We peeled off the small rollers on the omni wheels, and removed the inserts. Then, we started adding a high strength spacer inside, and then put it on a high strength axle in order for the tiny rollers to spin when powered. We didn't get to test it today, and will finish the full new assembly of the counter-rollers at a later time.



Wiring:

We started to look for optimal brain placement on the robot so that all ports are accessible, and that battery plugin will be efficient. We found a spot on the side of our robot that is relatively low to the ground for a lower center of gravity. The C Channel holes were impossible to fit a screwdriver in due to the cascade lift obstructing it, so we utilized collars and put them through the side of the C Channel for attachment points for the brain.



Appendix A: Github Repositories

Robot 1: “Frostbyte”

QR Code for Physical Notebook Readers



Link for Digital Notebook Readers:

<https://github.com/TNelson-code/12A-Frostbyte-26-27>

Appendix B:

External

Sources

Citations

All links in notebook entries are referenced and cited here. Each citation has the page number of the entry that it is mentioned in, and a link to the source.

Page	Source Name	Link
n/a	Game Manual	https://content.vexrobotics.com/docs/25-26/v5rc-push-back/docs/Push-Back-2.0.pdf
6	2654E's Notebook Formatting Video	https://www.youtube.com/watch?v=33GIOvKLITM